



Deep learning at the service of metaheuristics for solving numerical optimization problems

Olaide N. Oyelade¹ · Absalom E. Ezugwu² · Apu K. Saha³ · Nguyen V. Thieu⁴ · Amir H. Gandomi⁵

Received: 7 June 2023 / Accepted: 3 October 2024 / Published online: 23 January 2025
© The Author(s) 2025

Abstract

Integrating deep learning methods into metaheuristic algorithms has gained attention for addressing design-related issues and enhancing performance. The primary objective is to improve solution quality and convergence speed within solution search spaces. This study investigates the use of deep learning methods as a generative model to learn historical content, including global best and worst solutions, solution sequences, function evaluation patterns, solution space characteristics, population modification trajectories, and movement between local and global search processes. An LSTM-based architecture is trained on dynamic optimization data collected during the metaheuristic optimization process. The trained model generates an initial solution space and is integrated into the optimization algorithms to intelligently monitor the search process during exploration and exploitation phases. The proposed deep learning-based methods are evaluated on 55 benchmark functions of varying complexities, including CEC 2017 and compared with 13 biology-based, evolution-based, and swarm-based metaheuristic algorithms. Experimental results demonstrate that all the deep learning-based optimization algorithms achieve high-quality solutions, faster convergence rates, and significant performance improvements. These findings highlight the critical role of deep learning in addressing design issues, enhancing solution quality, trajectory, and performance speed in metaheuristic algorithms.

Keywords Benchmark functions · Deep learning · Long short-term memory, LSTM · Metaheuristic algorithms · Neural networks

1 Introduction

Computational solutions for addressing most real-life problems have become ubiquitous. However, most real-life problems are complex and present varying computational difficulties during the application, including long computational time, computational resource intensiveness, high bandwidth requirement, significant software development efforts, and memory-intensive computation. Researchers have focused on finding optimal solutions or automating procedures for solving these complex real-world problems. Several computational techniques have been introduced for effectively and optimally improving real-life applications of computational approaches to a domain of interest. Methods include mathematical modeling, heuristic algorithms, metaheuristic models, learning models, rule-based approaches, et al. Although most of these methods have

✉ Absalom E. Ezugwu
EzugwuA@ukzn.ac.za

¹ School of Electronics, Electrical Engineering and Computer Science, Queen's University, Belfast, UK

² Unit for Data Science and Computing, North-West University, 11 Hoffman Street, Potchefstroom 2520, South Africa

³ Department of Mathematics, National Institute of Technology Agartala, Agartala, Tripura 799046, India

⁴ Faculty of Computer Science, Phenikaa University, Yen Nghia, Ha Dong, Hanoi 12116, Vietnam

⁵ Department of Engineering and Information Technology, University of Technology Sydney, Ultimo, NSW 2007, Australia

been applied as a single solution to address practical problems, recent studies have promoted the need to hybridize different techniques for improved performance. For instance, the hybrid of metaheuristic algorithms and learning models has received wide research attention for solving real-life problems in areas such as engineering and medicine [1–3]. Moreover, these particular hybrids have demonstrated outstanding performance, thereby motivating further research [4, 7].

Deep learning (DL) encompasses a wide range of learning models that specialize in extracting meaningful patterns from image, text, and signal datasets. Within the field of deep learning, there are subfields such as reinforcement learning (RL), clustering analysis, and other related learning approaches. Notably, deep learning algorithms, particularly convolutional neural networks (CNN) and recurrent neural networks (RNN), have achieved remarkable success in various domains, including image analysis, text and image synthesis, and signal analysis [8–11]. While an RNN model has features of the general neural network, its input represents a sequence. Hence, the computation of the output includes states dependent on the memory of previous states. This sequential state's dependence limits the applicability of RNN to longer text input because remembering the items in older memories is difficult due to the vanishing gradient issue [12]. Long short-term memory (LSTM) was designed to address this shortcoming so that only useful states are remembered, thereby learning long-term dependencies. The gated recurrent units (GRU) advance this solution by adding a mechanism for states to forget. Interestingly, RNN, LSTM, and GRU have proven useful in time series prediction, predicting the next word of a text, sentiment analysis, machine translation, language and text synthetization, question-answering systems, classification tasks, and natural language processing. Characterized by the combinatorial problem of selecting optimal hyperparameters combination in deep learning, RNN and its variants have demonstrated increased performance when optimized using metaheuristic algorithms.

Metaheuristics algorithms can optimize procedures with complex computational solutions. These algorithms are designed based on some natural phenomena and are often categorized into evolution-based, mathematics-based, physics-based, human-based, swarm-based, music-based, and biology-based methods. Classical examples of such algorithms are the genetic algorithm (GA) [13], artificial bee colony (ABC) [14], particle swarm optimization (PSO) [15], and cuckoo search optimization (CSO) [16]. Recently, best-performing metaheuristic algorithms have been reported, including arithmetic optimization algorithm (AOA) [17], hunger games search (HGS) [18], equilibrium optimizer (EO) [19], Ebola optimization search algorithm

[20, 21] and its immunity-based variant referred to as the immunity-based Ebola optimization search algorithm [22]. Although these algorithms have been widely used for solving several real-life complex problems, their performance is often hinged on finding good initial parameter settings, the quality of initial solutions, achieving a balance between the explorative and exploitative search process, adaptability to solve particular problems, parameter turning, and functional evaluations (FEs). While the design of new algorithms is the primary focus of research in this field, the inherent challenges associated with the design and application of metaheuristic algorithms have received minimal attention. At the same time, little or no effort is channeled toward addressing these problems in the existing algorithms. Therefore, this study investigated high-performing computational techniques that are suitable for addressing the general problems of metaheuristic algorithms.

A multitude of studies have employed metaheuristic algorithms to optimize various computational techniques, including deep learning. These optimization efforts targeting complex real-life problems have showcased remarkable performance, as all existing optimization algorithms have yielded comparable results when applied to tackle such problems. Given this context, the pertinent research question arises: can the integration of deep learning methods enhance the performance of metaheuristic algorithms?

Recently, we found some motivation for using deep learning techniques in metaheuristic algorithm design. A previous study [23] outlines a plethora of current trends pertaining to the use of machine learning in metaheuristic design and reveals existential research gaps. For instance, the challenge of parameter fine-tuning in metaheuristic algorithms has been investigated using deep learning methods [24–26]. Another issue that has received attention in applying deep learning models to metaheuristic algorithms is the auto-design of problem-specific algorithms. For example, [26] applied grammar-aided genetic programming to design metaheuristic algorithms that are robust and efficient in handling optimization problems. Similarly, [27] showed the relevance of an automated binary optimization algorithm design model for obtaining suitable algorithms for solving binary optimization problems. A component-based method for designing optimization algorithms using features from the mathematics method and genetic algorithm methods have been reported in [28]. Another interesting research on the use of deep learning in metaheuristics is the concept of surrogacy, which allows a learning mechanism to learn the behavioral pattern of objective functions to improve the solver.

Interestingly, deep learning algorithms have been used as surrogate models for solving such regression problems. For instance, the surrogate method, which is trained on

sample points, has been used in the evolution algorithm (EA) to reduce the computational cost for functional evaluations (FEs) to improve global and local searches [29]. The same surrogate method was applied in [30] to improve the FEs for better exploration and exploitation search. Support vector regression (SVR) surrogacy is reported in [31], and a deep belief network (DBN) surrogacy was applied to the particle swarm optimization (PSO) algorithm in [32]. Other research efforts include the use of a statistical deep learning model, which is based on the transfer learning technique to generate initial solutions and evolve optimization algorithms for the dynamic multi-objective evolutionary algorithm (EA) [33]; the use of inductive learning techniques in metaheuristics [34]; and finding the best problem-specific algorithm from a pool of optimization algorithms that is suitable for solving the problem [35]. Unfortunately, all these studies failed to maximize relevant data generated by metaheuristic algorithms to improve integration with deep learning. The rich dynamic information generated by the metaheuristics, which carries historical content, includes information on the global best, worst and sequence of solutions obtained during each iteration, the pattern of use of function evaluations (FEs), solution space and trajectories of population modification, movement between local and global search processes, and many others. Utilizing the machine and deep learning models with such dynamic data holds potential for improving the performance and design issues in the conventional optimization framework to solve combinatorial optimization problems.

This study aimed to combine metaheuristics with deep learning methods to obtain—what we describe as—a neural combinatorial optimization solution to aid the design, speed, quality, and performance of metaheuristic algorithms. Specifically, the biology-based, evolution-based, and swarm-based algorithms investigated in this study include the EOSA [21], IEOSA [22], slime mold algorithm (SMA) [36], virus colony search (VCS) [37], invasive weed optimization (IWO) [38], genetic algorithm (GA) [13], differential evolution (DE) [39], aquila optimizer (AO) [40], firefly algorithm (FFA) [41], PSO [15], sparrow search algorithm (SSA) [42], HGS [18], and whale optimization algorithm (WOA) [43]. The investigation of the current study is to support the generation of very qualitative initial solutions and a robust search process so that the final optimization process's final solution is of high quality and suitable for addressing the underlying optimization problem. Meanwhile, it is expected that this will improve the convergence rate and robustness of the algorithm, as observed in [44]. The use of LSTM, a type of neural network architecture, is proposed for learning important patterns from dynamic data collected during the iteration process on the metaheuristic algorithms. The trained model

is then applied to support the generation of initialization solutions and the guided search process. This advances research beyond the basic initialization methods, namely random number generator [45], quasi-random [46], adaptive randomness [47], support vector machine based [48], chaos theory, Lévy [49], Latin hypercube, and Kronecker [50], for improved qualitative solution differentiation and reduced computational costs [23].

The dynamic information employed in this study's experimentation is not contingent on the nature of the specific problem or any particular dataset. Instead, we introduced and utilized classical benchmark functions along with the CEC functions. These were employed initially to execute the metaheuristic algorithm, following which we captured this dynamic information for the training of a deep learning model. Using the long short-term memory (LSTM) architecture, this model comprehends patterns and adjusts to generate a synthesized solution space based on neural networks. This synthesized space serves as a generative model, improving the development of a high-quality search space to streamline the optimization process. It is important to note that aspects like the number of local minima, the complexities of the objective function's shape and pattern, and considerations related to exploration, exploitation, and individual mutation are beyond the scope of this study. Our primary focus revolves around introducing a deep learning approach that centers on optimizing the initialization of potential solutions, tailor-made for compatibility with metaheuristic algorithms. The technical contributions of this study are outlined as follows:

- a) Formulation of a DL technique that enhances the initialization process of metaheuristics by constructing a complete solution from an empty one. In this context, the new solution generation method replaces the conventional randomized initialization strategies.
- b) Implementation of an online learning strategy that facilitates dynamic knowledge extraction and its application in generating initial solutions for a given problem instance. A notable advantage of this approach is that, while the extracted knowledge may not be highly extensive, it perfectly aligns with the specific characteristics of the current problem instance.
- c) The design of a pool of metaheuristic algorithms implemented to gather and formalize dynamic data throughout the search and optimization process. This also include the implementation of DL-based metaheuristics.
- d) A neural network architecture is built and trained based on LSTM for learning trajectory of solution space and sequence.

- e) Assessing and validating the performance of the knowledge-aware or DL-based metaheuristic optimization techniques across sets of challenging mathematical benchmark functions and CEC 2017 functions. Likewise, presenting a comprehensive comparative analysis study to demonstrate the superiority of the proposed methods.

The remaining part of this paper is structured as follows. In Sect. 2, a review of similar studies that have applied deep learning to metaheuristic algorithms and the use of the neural network for text generation is presented. A detailed presentation of the proposed methodology in the design of the neural computational optimization learning model is given in Sect. 3. Sections 4 and 5 discuss the experimentation and results of the proposed approach. The study is concluded in Sect. 6.

2 Related work

A detailed review of related studies on the subject of applying deep or deep learning in metaheuristics algorithm design is presented in this section, based on the core component of the proposed method. The sub-sections describe studies on the application of deep learning to the design and fine-tuning of metaheuristics and the use of deep learning in generating text. A reviewed summary is thereafter given to comparatively describe the current achievement in the literature on the subject area and to motivate for the new approach.

2.1 Application of deep learning to design metaheuristic algorithms

Automatic design of an optimization algorithm is a problem-centered approach for developing a metaheuristic algorithm suitable for addressing a particular problem. Hughes et al. [51] addressed the black-box optimization problem by investigating the viability of grammar-guided genetic programming to obtain an optimization algorithm for solving the problem. The rule-based approach was combined with a particle swarm optimization (PSO) configuration so that the hybrid model utilizes some building blocks of such a class of algorithm to realize the most suitable algorithm. The procedure for obtaining the desired algorithm is determined by experimentation with the hybrid model in a search space, where the population computationally evolves in solving the optimization problem. A suitable algorithm is selected based on the quality of the resulting population achieved after some runs. Recently, Souza proposed [27] a similar method, solver, and code-named AutoBQP to solve binary optimization

problems using a search for the best heuristics method. The method automates the process of searching and developing an algorithm that is suitable for solving the defined problem. Pessoa et al. [28] was motivated by the latter research on automating metaheuristic algorithms and aimed to generate algorithms capable of yielding high-quality solutions for a defined problem with time constraints. The optimization algorithm generating approach was investigated for multi-level capacitated lot-sizing problems in production planning. Following a component-based method for the design of optimization algorithms, the study adopted features from both mathematics and genetic algorithms. Results showed that the approach yielded good optimization algorithms for the problem.

Using a different approach to metaheuristic algorithm design, Nakib et al. [52] proposed a maximum likelihood estimation and mutual information framework. The proposed framework leverages a component-based procedure in the design process to apply a deep learning model to drive the metaheuristic algorithm design process toward the problem-solving aspect. Although we consider this approach interesting, the deep learning model could be used to investigate the algorithm evolution process and not just the problem-solving process. Interestingly, Jiang et al. [33] demonstrated the workability of this idea by developing an approach to solve problems associated with dynamic multi-objective optimization problems (DMOPs). The study aimed to generate suitable metaheuristic algorithms for solving such problems using a bank of experiences in the domain to train a statistical deep learning model, which can automate the process. The deep learning approach uses the transfer learning technique to leverage the databank of experiences in similar domains to generate initial solutions and evolve optimization algorithms for a dynamic multi-objective evolutionary algorithm (EA). Experimental procedures were applied to evaluate the proposed method using non-dominated sorting genetic algorithm II, multi-objective particle swarm optimization, and the regularity model-based estimation of the distribution algorithm.

The need to ensure efficient parameter tuning for metaheuristic algorithms has been motivated as another approach to automate the design of algorithms in this category. For instance, Zennaki and Cherif [24] solved challenging combinatorial problems using deep learning-enabled metaheuristic algorithms. The deep learning approach uses a trained decision rule algorithm to predict the optimal solution for solving the combinatorial optimization problem. A Corpus dataset was collected and used to train the deep learning model so that classification rules were extracted to support the fine-tuning parameter process of the metaheuristic algorithm. In a related study, the author contributed to improving the automation for

designing metaheuristics using neural networks and some other techniques. These methods were applied to evolve the parameter tuning to simplify and standardize the metaheuristic. The neural network approach was combined with the design of the experiment's method for obtaining initial parameter settings for metaheuristic algorithms [53]. Garmendia et al. [54] described the use of neural network models in the design of metaheuristic algorithms as a neural combinatorial optimization algorithm design. This description embodies the use of machine and deep learning and other associated methods in supporting the design process of optimization algorithms for combinatorial problems. The workability and performance of the method were investigated using the linear ordering combinatorial optimization problem.

On the other hand, another study considered improving the generalization of metaheuristic as a necessary contribution toward automating algorithm design. The study designed search operators, a relevant component of metaheuristic algorithms, using modified gene expression programming (GEP) that applied the multi-criteria technique to obtain the best combination of search operators capable of solving an optimization problem with increased accuracy [55]. Meanwhile, other studies have proposed the need to add a surrogate model to existing metaheuristic algorithms to achieve better performance. Mejía-de-Dios and Mezura-Montes [56] conceived the notion of using two surrogate methods based on kernel interpolation numerical to support metaheuristic algorithms in solving what is known as a bilevel optimization problem (BOP). BOP problems are considered challenging since an optimum solution for the upper-level problem must be obtained in order to solve the lower-level problem. The study's outcome showed that the two surrogate methods successfully approximated both the upper- and lower-level objective functions when used with a population-based metaheuristic.

Lessmann et al. [25] used data mining techniques to address the issue of parameter tuning of the optimization algorithm. The study applied the fine-tuned parameters to improve the performance of the simulated annealing metaheuristic algorithm so that the problem of the cooling factor associated with the algorithm is addressed. The deep learning approach used for the parameter fine-tuning is comprised of verified regression models that are applied to learn suitable parameter values that are then used on particle swarm optimization (PSO). In a related study, Huang et al. [26] described a detailed review of studies that investigated the practice of using learning models to find the best combination and values of parameters suitable for improving the performance of metaheuristic algorithms. The authors noted that the fine-tuning parameter techniques found in the literature include automated methods, which

can be categorized into simple-generate-evaluate, iterative-generate-evaluate, and high-level generate-evaluate methods. The study presented potential researchers in the domain and the benefits of using each method. Chen et al. [57] proposed a deep learning-based approach to solve the difficult problem of unconstrained binary quadratic programming (UBQP) based on deep reinforcement learning (DRLH). Supported by a neural network model, the proposed DRLH model selects the optimal variable each time a solution is developed. Meanwhile, two variants of DRLH, namely simplified DRLH (DRLS) and DRLS with hill climbing (DRLS-HC), were investigated in the study. Results showed that the proposed DRLH-HC yields better high-quality solutions, followed by the DRLS algorithm.

Fan et al. [29] applied the surrogate method to an evolution algorithm (EA) through the use of large sample points, which reduced the computational cost for functional evaluations (FEs). The learning process for the surrogate method uses ensemble learning trained on the collected sample points so that the resulting model yields support for both global and local searches. A clustering approach was also proposed to learn the structure of the Pareto optimal set for solving multi-objective optimization problems (MOPs). The aim is to evolve a better-performing optimization algorithm for solving multi-objectives by using the learned structure from the clustering procedure to evolve elite offspring for the evolution optimization algorithm. Supported by the Gaussian-perturbed solution initialization process, the sampling strategy using differential evolution (DE) operator, a reusing scheme, and an adaptive strength Pareto approach were all hybridized for reduced computational cost, balancing exploration and exploitation, and collection of information on local and global operations [58]. Liu et al. [30] demonstrated the usability of a quasi-affine transformation evolutionary (QUATRE) algorithm to perform the role of surrogacy assistive (SA) in improving the performance of the optimization algorithm. The use of SA-QUATRE follows the global and local surrogate models that were applied to improve the function evaluations (FEs) component of optimization algorithms. The two surrogate models were trained using the information collected from the metaheuristic algorithms' exploration and exploitation search processes, which corresponds with the roles played by the local and search mechanism of the algorithms. Using an evolution-based optimization algorithm, the study leveraged generation and individual-based evolution control to enrich the surrogacy solution.

Oliveira et al. [31] reported another deep learning-based surrogate solution for improving metaheuristic algorithms and investigated the use of different deep learning models for the task of surrogacy on metaheuristic algorithms. The trained models, including decision trees, random forest,

and support vector regression (SVR), were trained under varying conditions. The trained model was then applied to improve the performance of optimization algorithms. In a related study, Tao and Sun utilized a special type of surrogate mechanism for metaheuristic algorithms. The multi-fidelity surrogate method was compared with a deep belief network (DBN), which was trained using the k-step contrastive divergence technique to achieve better prediction performance. The learning model was trained using high-fidelity data so that the resulting model was used for prediction purposes. Both the trained DBN and multi-surrogacy methods were applied to improve the performance of metaheuristic algorithms. Results showed that the latter method performed better than the former, confirming the relevance of the surrogacy in the problem domain when used with the PSO algorithm [32].

Studies that have addressed the challenge of auto-generating, surrogate-supporting, and parameter fine-tuning related to metaheuristic algorithms have been reviewed in previous paragraphs. However, we found other studies that focused on investigating the task of finding optimal algorithms from a pool of metaheuristic algorithms to solve specific problems. Gutierrez-Rodríguez et al. [35] demonstrated this by using a meta-learning technique to select an optimal metaheuristic algorithm suitable for solving vehicle routing problems with time windows. The deep learning model was trained by first formalizing features representing the domain knowledge to optimize such features using an optimization algorithm. The trained model was then applied to predict the best metaheuristic. A multilayer perceptron classifier was appended to the trained model for classification purpose, and the popular wrapper selection method was used for the meta-feature selection. Wauters et al. [59] reported that reinforcement learning is also a suitable deep learning method for improving the performance of metaheuristic algorithms. The experience-based reinforcement learning method relies on learning from some data representing past experiences that can be sufficiently trained to generate metaheuristic algorithms suitable for addressing scheduling problems. The study was also motivated by using reinforcement learning techniques as a performance booster for metaheuristic algorithms. Calvet et al. [60] emphasized the usefulness of deep learning algorithms to support the input model used by the metaheuristic with the general aim of performance enhancement.

We note that the use of machine learning models, such as classifiers, reinforcement learning, and all other learning techniques have shown that surrogacy methods for performance improvement in metaheuristic algorithms have been widely investigated. While all these methods have yielded interesting results, they only serve as inspiration for future studies that are interested in widening the research

scope beyond deep learning to other aspects of learning. This study intends to present one of such advanced learning methods for enhancing the search process and solution generation of metaheuristic algorithms. In the next section, we present a review of the achievement of those learning models in other domains and then motivate for its use in the context of the problem discussed in the study.

2.2 LSTM-based text generation

Text generation has proven useful in several domains where automation of text synthetization is required. The use of the LSTM model for this task has proven relevant and is, therefore, reviewed in this section. This is necessary to understand the approach to support the readership of similar approaches in this study.

Dhall et al. [9] proposed a stacked LSTM model for text synthetization using random seed that can predict the number of characters required for text, which it is expected to generate. This approach demonstrates some advancement compared with most studies that often do not use the stacking method in LSTM. In another interesting manner, LSTM has been reportedly used for domain name generation as an attempt to curtail network attacks. For instance, the model was applied to analyze the flow of algorithms for generating domain names known for outputting alphanumeric associated with domain names. The trained model successfully learned to identify the pattern that the algorithm uses to generate domain names [61]. Another interesting use of LSTM is as a generative model for electrocardiograms. This was achieved using a hybrid of the generative adversarial network (GAN), CNN, and LSTM, and named BiLSTM-CNN. The synthesized ECG data were used to support the diagnosis of patients with heart disease [8]. Wen et al. [62] presented a similar hybrid of LSTM with a statistical language generator to generate natural language. A similar study proposed embedding a context vector into LSTM for natural language text generating purposes [63]. In a related work, LSTM was used to generate text based on the Bangla language, demonstrating a kind of natural language generator [64]. Another study utilized LSTM for generating texture, also known as exemplar-based texture synthesis, to support image processing [65].

In addition to predicting next character in text generation, the LSTM model can also be utilized in forecasting the next event based on a given input. Rauf et al. [66] leveraged the optimized temporal component and nonlinearity of LSTM to forecast the spread of COVID-19 to reduce the mean absolute error of the computation. Still on the issue of forecasting, Bouktif et al. [67] applied LSTM to electric load forecasting to support the management of power grid networks. The study hybridized two

metaheuristic algorithms, namely genetic algorithm (GA) and particle swarm optimization (PSO), with LSTM to optimize hyperparameter selection for better forecasting performance. Similarly, Alkabbani et al. [68] reviewed the performance of RNN and other associated deep learning models in energy forecasting. Stock market forecasting, which predicts future trends in stock market activities, was also studied using LSTM models. Metaheuristic-optimized use of the RNN-LSTM hybrid was employed for intra-day stock market prediction. The flower pollination algorithm and PSO were used to design two optimized models of RNN-LSTM hybrids [69].

Data streaming is an aspect of computing that can benefit from using RNN-LSTM models to predict the following stream of data in the pipeline. Kumar and Batra [70] investigated the performance of RNN, LSTM, and GRU models in predicting the succeeding data sequence in a stream. The authors also optimized hyperparameters of GRU and RNN models using a GA to determine if the optimization yields better results. Although most studies on LSTM for generative purposes have focused on text-based synthetization, we found other studies that used the model for music generation. Conner et al. [71] reported successfully using LSTM to generate music. Qiao et al. [72] combined CNN with LSTM to learn features of a computer network mesh and synthesize a near-real 3D mesh network. Text generation using deep learning models is an interesting domain, which now has broad applicability beyond text-related applications. Currently, data streaming, music generation, and network domain name areas now use the models for information synthetization. Motivated by this, the presented study considered using the LSTM model for predicting suitable solutions for population initialization to improve the performance of population-based metaheuristic algorithms.

The review presented in this section has revealed the significant efforts of using both metaheuristic algorithms and deep learning algorithms to solve a wide range of domain-related problems. The hybridization of the two algorithms was presented in this section with emphasize on studies using metaheuristic algorithms to improve the performance of deep learning models and the use of deep learning models to improve performance of metaheuristic algorithms. Although the former approach has received more research focus, the latter still presents interesting research opportunities in designing optimization algorithms, which demonstrate some measure of intelligence starting from the initialization phase into the search operation phase. This study aimed to investigate the possible

performance enhancement of integrating deep learning models into the metaheuristic algorithm design process. The next section provides detail on the methodology for this proposal.

2.3 Limitations of existing methods

The limitations of employing deep learning models in the context of metaheuristic optimization algorithms for solving numerical optimization problems include challenges related to the performance of DL-based approaches. This can be attributed to the inadequacy of the generated datasets used for the training of these models as evidence in some of the findings report in the current work. Additionally, the potential limitation also lies in the richness of the extracted knowledge, which may not be extensive enough. Despite these constraints, several advancements are being reported in the literature, and innovative strategies such as the online learning utilized in this study and DL techniques for metaheuristic initialization show promise in overcoming some of the aforementioned limitations [44].

3 Proposed research methodology

The methodology designed for this study is presented and detailed in this section. A general overview of the approach is first presented, describing the model of the neural combinatorial optimization process for metaheuristic algorithms. Data generated during the optimization process of metaheuristic algorithms were collected for use in training the deep learning model. Following the approach overview, this dynamic data collection method is described. The design of the neural network used for the heuristic learning phase and the trained model's application to improve the workings of the metaheuristic algorithm are also discussed.

It is noteworthy that this research is driven by the desire to harness the learning abilities of neural architectures in aiding an intelligently directed search process and the initialization of populations for metaheuristic algorithms. Our study introduces a novel approach by recognizing the numerous methods suggested in literature for initializing search spaces, including random and chaotic methods. We advocate for a 'sensual' approach, one that comprehends the intricacies of a search algorithm and provides it with valuable candidate solutions within the search space.

3.1 Approach overview

The use of deep learning methods to learn heuristic data for improved performance of metaheuristic algorithms demand a clear integration of methods. This integration must be done in a manner that suggests performance gain for the metaheuristic algorithm and does not underperform the natural optimization process inherent in the algorithm. Considering this, a model representing the neural network-based performance enhancement method for the metaheuristic algorithm is shown in Fig. 1. The design of the model describes well-integrated components, which seamlessly cooperate to achieve the aim of the desired performance gain.

Figure 1 presents the optimization model, comprising four main components: an iterative optimizer, a pool of metaheuristic algorithms, dynamic data collection and formalization, and deep learning model design. The pool of metaheuristic algorithms is divided into three categories: evolution-based, biology-based, and swarm-based. Evolution-based algorithms include the GA and DE. Biology-based algorithms consist of EOSA, SMA, IWO, and VCS.

Swarm-based algorithms include PSO, WOA, AO, HGS, SSA, and FFA. The figure depicts two designs for the iterative optimizer: one for non-guided optimization using metaheuristic algorithms and another for deep learning-guided optimization. Throughout the non-guided optimization iteration, dynamic data are generated, collected, and formalized for storage, serving as training data for the deep learning model.

As illustrated in Fig. 1, the process flow incorporates three types of data storage: formalized dynamic data storage, a repository of trained models for each metaheuristic algorithm category, and storage for potential solutions generated by the deep learning model. Each storage point is strategically connected to the process flow to ensure coherent data flow. For example, formalized data are retrieved from storage for preprocessing before building the neural network. These collected data are cleaned and preprocessed to train the deep learning model. The LSTM architecture, utilized in this study, is constructed and trained using a partition of the preprocessed data, with each trained model stored in the repository of trained models. The trained model generates solutions, which are stored in

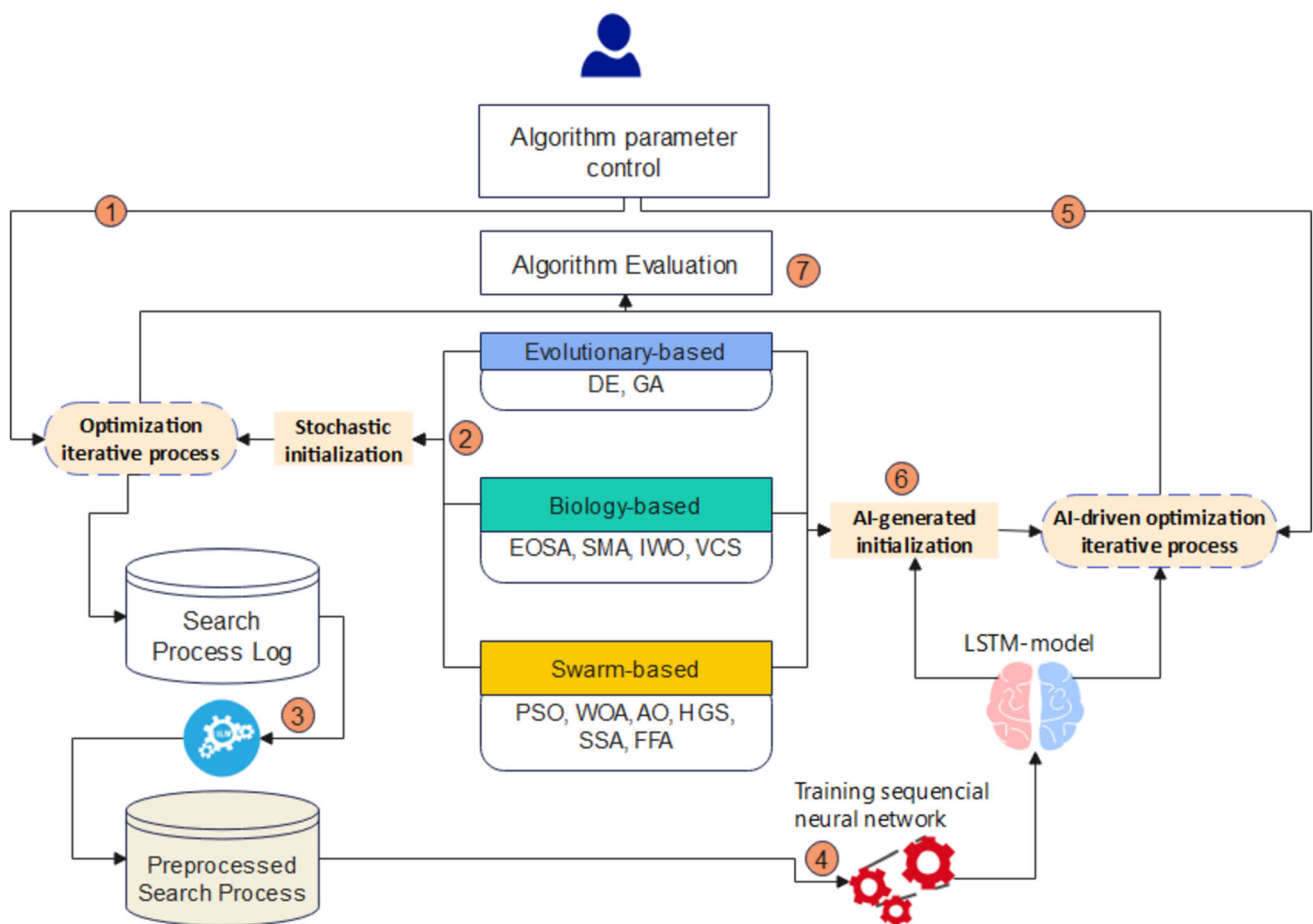


Fig. 1 A neural combinatorial optimization model for deep learning-enabled solution initialization and guided search process

the potential solutions database for each optimization algorithm. The LSTM-assisted iterative optimizer combines the trained model and solution space configuration, following a natural evolution procedure to find the best solution for optimization problems. The performance of the two iterative optimizers is then compared and reported. Importantly, the framework depicted in Fig. 1 maintains the intrinsic behavior of metaheuristic algorithms without disruption. It does not hinder the exploration and exploitation search processes of these algorithms, including PSO, WOA, EOSA, GA, DE, IWO, FFA, and others used in the experimentation. The interaction between our approach and metaheuristic algorithms can be better understood by referring to Fig. 1, which visually demonstrates how our approach coexists harmoniously with these algorithms, preserving their inherent exploration and exploitation dynamics.

3.2 Dynamic data collection and formalization

The iterative search process of optimization algorithms is typically known to generate dynamic data. These data include the trajectory of solutions, movement between the local and global search, global best and worst solutions for each epoch, solution in the objective space, and other vital operations. This study utilized deep online learning-supported metaheuristics to approach the data gathering process. Dynamic data are gathered when the optimizer solves the problem. This approach differs from the cold form or offline method that collects such data before the solving phase. Figure 2 provides an illustration of how the online, also known as hot form, data gathering generates training data for the deep learning models. The optimization is placed in the iterative optimizer. Three data types are collected for each iteration when the search and update mechanism has applied its operations in the solution space.

The dynamic data, which include the current solution space, elite and bad solutions, are collected for each iteration and then serialized for storage. Meanwhile, since a complete predefined iteration number must be completed to

terminate the algorithm, the collected data are buffered after the serialization operation, which allows for the formalization of the data for effective storage. These data collections are stored in an algorithm-specific format according to their categories so that data for the swarm-based algorithms are differentiated from biology-based algorithms. Recall that these are necessary for seamless data integration into the neural network models described in the next subsection.

3.3 Neural network architecture

Text-based data generation or synthetization has proven to be more effective for RNN over CNN, which is popular with image synthetization [73]. Moreover, to circumvent the vanishing problem associated with RNN, we applied an LSTM architecture for the neural optimization task. A summarized description of an LSTM cell is presented in Fig. 3, displaying the input into the cell as x_t using three *sigmoid* and two *tanh* functions for activation. This cell description for the LSTM is often used to build an LSTM layer consisting of a specified number of neurons.

The architecture of the proposed neural network involves the stacking of LSTM layers with other types of neural network layers. Figure 4 illustrates the architecture of the LSTM model. Prior to being passed to the first layer, known as the embedding layer, the inputs undergo pre-processing for data cleaning and formatting. This pre-processing step is necessary due to the numeric and non-alphanumeric nature of the inputs. It is important to note that the inputs include global best and worst solutions, as well as solution trajectories, which naturally consist of floating-point numbers. Therefore, the data are represented in a way that allows both the data input and corresponding labels to be extracted and normalized for input to the LSTM architecture. Additionally, the vocabulary of the input is extracted and computed from the raw data, supporting the class-labeling operation.

The proposed neural network model comprises two layers of LSTM, followed by a dropout layer and batch

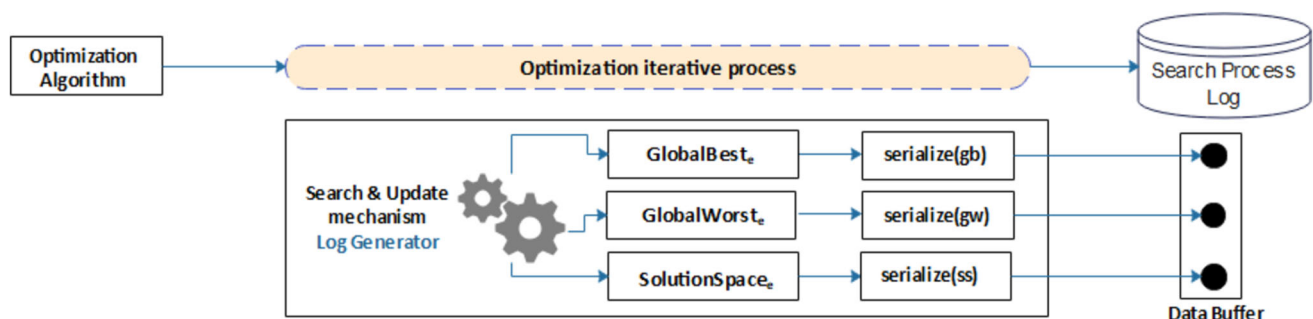


Fig. 2 Visualization depicting the dynamic data generation and formalization of the optimization algorithm for its application to the deep learning model

Fig. 3 A detailed representation of LSTM cell

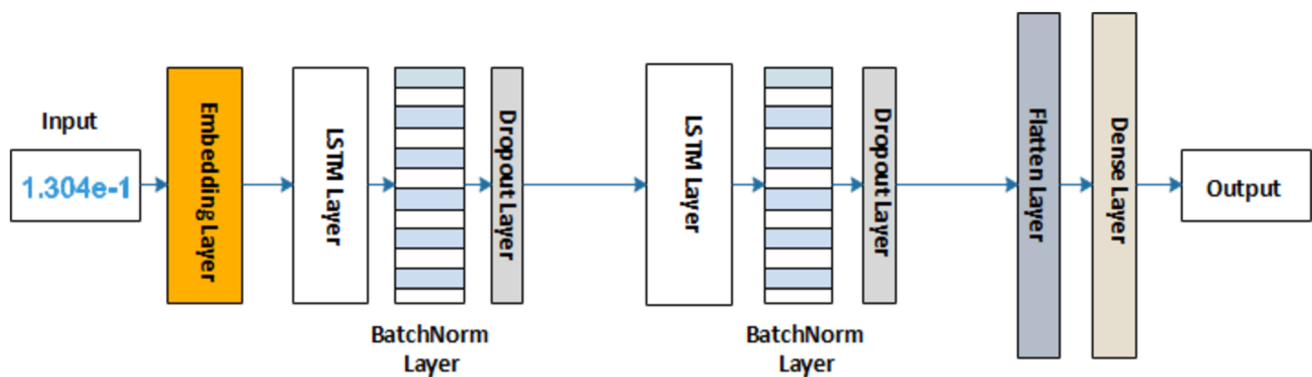
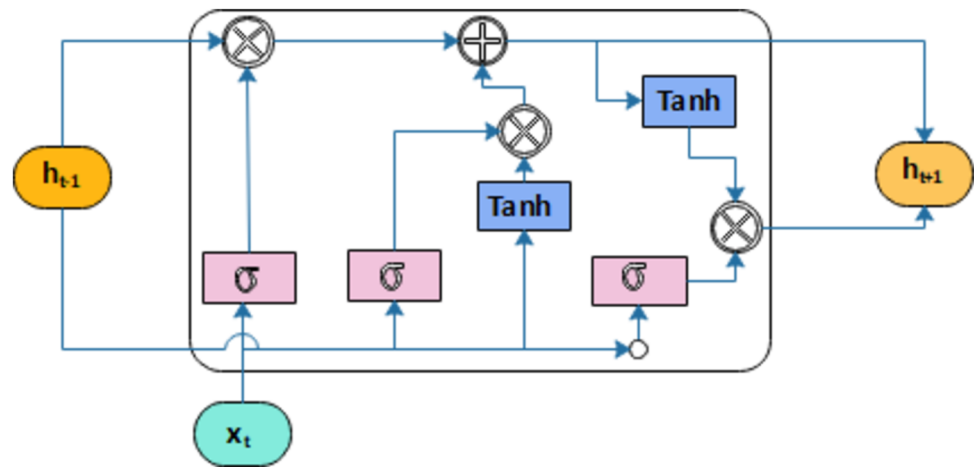


Fig. 4 Proposed LSTM architecture

normalization layer. The dropout layer reduces the number of features from the LSTM layer, aiding in better generalization of the model. The batch normalization layer ensures that the trained model stabilizes the learning process. To obtain the output from the model, flatten and dense layers are added after the two blocks of LSTM layers.

The value of 0.1 was used for all dropout layers applied to the LSTM blocks, and each layer of the LSTM was designed with 1024 neurons. The glorot uniform method was used for the recurrent initializer in each LSTM layer. The model's training was evaluated using the Adam optimizer, and a categorical cross-entropy loss function was used for the training. The model was trained over an epoch for the three categories of metaheuristic algorithms in order to investigate the effect of the data from each category on the model's performance. The trained models were then applied for optimization and search process of the metaheuristic algorithms, as discussed in the next subsection.

3.4 Neural network-guided initialization and search process

Enhancing the optimization process of metaheuristic algorithms using deep learning models is the core component of this study. This was achieved by the trained neural network model that is capable of generating initial solutions for each optimization algorithm investigated in this study. Also, the trained model was applied to support the search process so that the optimization process can output a better solution than the classical optimization method. The trained models for the evolution-based, biology-based, and swarm-based methods were used for generating raw solutions R , which is parsed as seen in Eq. (1), for regularization with the requirement of the optimization algorithms using D dimension. The outcome of this operation is a well-parsed solution for input to the metaheuristic algorithms.

$$R = \text{parse}(R, D) \quad (1)$$

The parsed initial solution is listed for N items, which correspond with a predefined population size for the metaheuristic algorithms. In Eq. (2), each individual s_i in the initial population, R , is extracted in a pattern to distinguish all s_i in S :

$$S = \left\{ s_i \in R \oint d | 0 \leq i \leq N \right\} \quad (2)$$

where $\oint$ is an N -ary operator with a clockwise integral operation on R , and d represents the delimiter that indicates the end and beginning of the previous s_{i-1} from next s_i individual in the population. In Eq. (3), all s_i in S are applied as positions representing the initial solution X :

$$X = \{x_i \in \text{position}(S, i) | 0 \leq i \leq N\} \quad (3)$$

The objective is to modify the solution space iteratively throughout the optimization process. Given that each metaheuristic algorithm incorporates distinct amendment procedures, like the social and individual behaviors in PSO or the gradual reduction in temperature and step sizes over time in SA, we have employed Eq. 3. This choice of equation serves as a versatile formalism capable of accommodating these unique variations. Its implementation enables us to apply our method across a range of metaheuristic algorithms.

Meanwhile, X is used as the initial solution set for optimization algorithms, and x_i in X is amended, as seen in Eq. (4), within the upper (ub) and lower (lb) bounds of the fitness function being considered in the optimization process:

$$x_i = [\text{amend}(x_i)]_{lb}^{ub} \quad (4)$$

The amended and final representation of X is presented as Eq. (5):

$$X = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,d-1} & x_{1,d} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,d-1} & x_{2,d} \\ \vdots & \vdots & & \vdots & \vdots \\ x_{i,j} & & & & \\ \vdots & & & & \\ x_{n,1} & x_{n,2} & \cdots & x_{n,d-1} & x_{n,d} \end{bmatrix} \quad (5)$$

The new framework offers a versatile solution that seamlessly integrates with population-based metaheuristic algorithms, providing guidance for initializing the search space. Equations 3, 4, and 5 are intentionally designed to possess a generic nature, applicable to various metaheuristic algorithms such as particle swarm optimization (PSO) or simulated annealing (SA). These equations capture shared characteristics found across different

metaheuristic algorithms. For example, Eq. 2 demonstrates how we leverage a deep learning architecture (LSTM) to derive mapped candidate solutions and incorporate them into the set S . In this equation, the symbol $\oint$ represents an N -ary operator employing a clockwise integral operation on R , while d serves as the delimiter, marking the transition between the previous s_{i-1} and the subsequent s_i individuals within the population. It is important to note that our framework allows for necessary adjustments to the solution space throughout the iterative phases of the optimization process. Equation 3 serves as a unifying framework, accommodating the distinct amendment operations intrinsic to each metaheuristic algorithm, such as social and individual behaviors in PSO or temperature reduction with diminishing step sizes over time in SA. This formalism enables the application of our approach to diverse metaheuristic algorithms, accommodating their specific variations.

During the optimization process for each iteration, the metaheuristic algorithm is expected to move between the exploration and exploitation phase to generate new nwx_i , replacing x_i in X , as seen in Eq. (6). Meanwhile, the predicted replacement for x_i is also obtained using the trained model in Eq. (7) to get px_i based on the global best obtained for the corresponding iteration number. The sequence of X is expected to transform over a number of iterations so that the final solution differs from the initial solution:

$$nwx_i = \text{localglobal}(x_i) \quad (6)$$

$$px_i = \text{predict}(gbest_i) \quad (7)$$

In Eq. (8), $gbest_i$ is compared with nwx_i to determine if it is suitable to replace x_i in X ; otherwise, px_i is used as the optimized replacement:

$$\text{optimized}_{x_i} = \begin{cases} nwx_i, & gbest_i < nwx_i \\ px_i, & \text{otherwise} \end{cases} \quad (8)$$

The operation procedure discussed in previous sections is further formalized using Algorithm 1 for implementation and formalization purposes.

Algorithm 1: Neural optimized metaheuristic algorithm

```

1  Definition:  $\varphi$  (MHAs): metaheuristic algorithms
       $f$ : trained neural network model
       $N$ : population size
       $tsplit$ : ratio of splitting between training and testing dataset
      result: an array of comparative results between two approaches for all algorithms

  Input:  $N$ ,  $MaxIter$ ,  $tsplit$ 
2  Output: results
3
4   $\varphi \leftarrow \{A_{psa}, A_{woa}, A_{ao}, A_{hgs}, A_{ssa}, A_{ffa}, A_{eosa}, A_{iwo}, A_{sma}, A_{vcs}, A_{de}, A_{ga}\}$ 
5   $X = stochastic(N)$  #initialization of search space
6  for  $m$  in  $\varphi$ 
7       $i = 0$ 
8      while  $i < MaxIter$ 
9           $X^{new}, fits_X, data \leftarrow m.optimize(X)$  #optimization process with data logging
10          $i++$ 
11      end-while
12       $t = \lfloor (size(data) * tsplit) / 100 \rfloor$  #data splitting for train/evaluation
13       $f = LSTM.fit(data[:t])$  #training the neural network model
14       $S = f(N)$  # using eq. (2) #AI-aided search space initialization
15       $S =$  position  $S$  using eq. (3)
16      amend  $S$  using eq. (4)
17      while  $i > 0$ 
18           $S^{new}, fits_S \leftarrow m.optimize(X, f)$  # applies eq. (6), eq. (7), and eq. (8)
19           $i--$ 
20      end-while
21       $result \leftarrow compare(fits_X, fits_S)$ 
22  end-for
23  return results

```

The algorithm takes several inputs, including the dataset split value, the number of epochs/iterations for training the optimization algorithms, the population size N , the trained model, and the pool of metaheuristic algorithms investigated in this study. The expected output of the algorithm is a compilation of all the optimized final solutions obtained from the metaheuristic algorithms. In the algorithm, the initial solution X is generated on line 5 and used during the optimization process for each algorithm, as iterated on lines 6–22. The operations specific to the individual iterative optimizers are performed on lines 8–11 and 17–20. Line 14 applies the trained model to generate the initial solution, while the guided search process is described on lines 17–20. Once all the metaheuristic algorithms have been experimented with, the results are returned on line 23.

To analyze the computational complexity of Algorithm 1, an algorithmic computational analysis was conducted, revealing a complexity of $O(n^2)$. This analysis primarily focuses on the computational demand for training each metaheuristic algorithm. Lines 8–11 of Algorithm 1

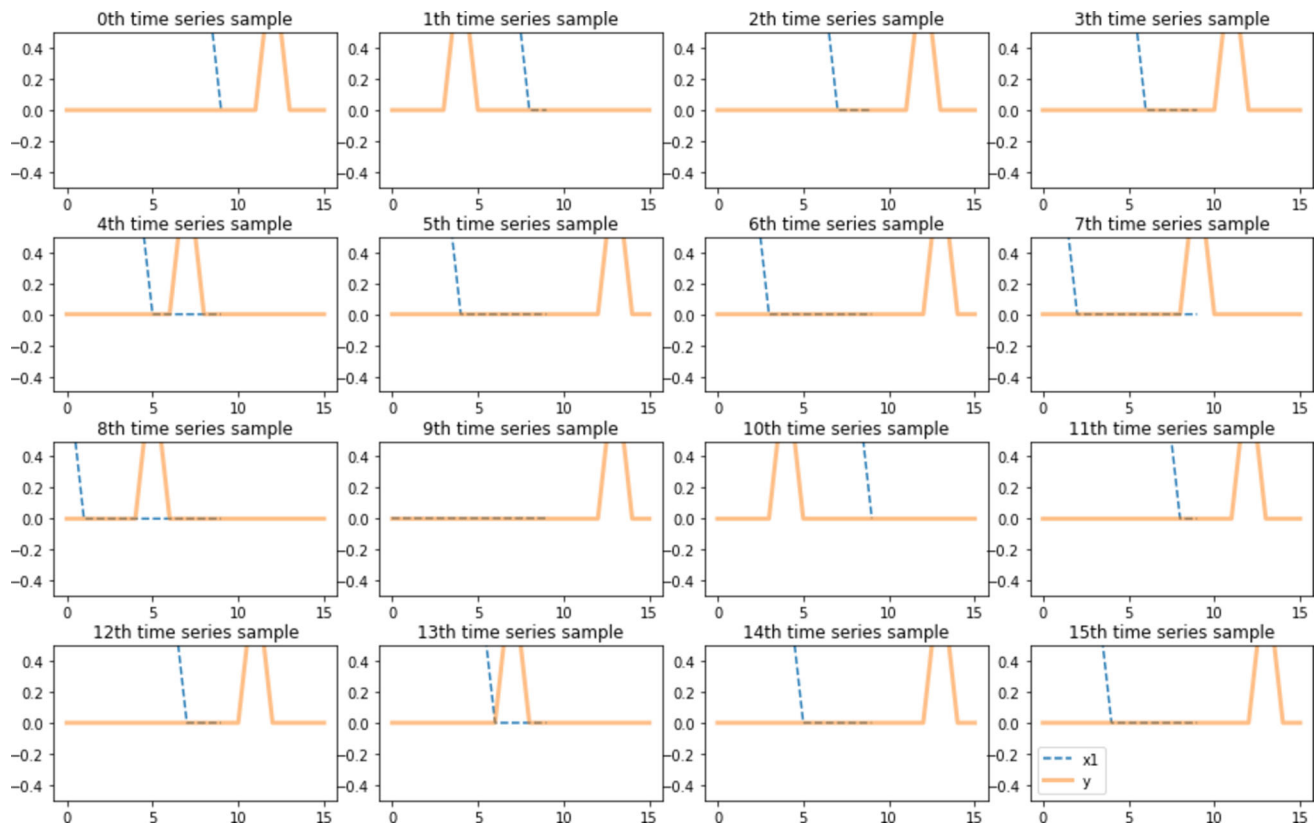
indicate that the computational analysis of each line is equivalent to $O(n)$. Similarly, each line between lines 17–20 can be clearly evaluated as $O(n)$. However, this analysis does not include the computational cost required for training the deep learning model, which trains for M epochs. This training process can be interpreted as having a complexity of $O(m)$. Assuming $N = M$, the complexity of running each metaheuristic algorithm is $O(n^3)$. The implementation of this algorithm is described in the next section, where the datasets are utilized for experimentation.

4 Experimental setup

In this section, we provide an overview of the experimental setup for evaluating the methodology outlined in the previous section. We outline the computational framework and offer further insights into the dataset utilized for the study, which includes the presentation of sample examples. Furthermore, we define the benchmark functions utilized

Table 1 Definition and value assignment for all parameters used for experimentation

Notation	Definition	Value	Usage
e	Deep learning model training epoch	50	LSTM model
Iter	Metaheuristic training number iteration	500	Metaheuristic algorithms
tsplit	Split ratio for training data for the LSTM model	> 0.9	LSTM model
BS	Batch size for training the LSTM model	10	LSTM model
N	Number of individuals in a population	100	Metaheuristic algorithms

**Fig. 5** A visualization of data used for the training of LSTM model

for evaluating the performance of all the investigated metaheuristic algorithms.

4.1 Computational environment configuration

The experiment for the optimization process of the metaheuristic algorithms was conducted on a personal computer (PC) with the following specifications: Intel (R) Core i5-4210U CPU 1.70 GHz, 2.40 GHz CPU; 8 GB RAM; and Windows 10 operating system. The training of the neural network was performed on a separate computer with the following configuration: Intel (R) Core i5-4200 CPU 1.70 GHz, 2.40 GHz CPU; 16 GB RAM; and a 64-bit Windows 10 operating system. The same system configuration was used for generating sample solutions using the trained model. The implementation of the LSTM architecture and the metaheuristic algorithms was accomplished

using Python 3.7.3, along with supporting libraries such as Numpy, Tensorflow, Keras, Mealpy, and other dependent libraries.

4.2 Parameter settings

Table 1 presents the parameters utilized in the experimentation involving the deep learning model and the metaheuristic optimization algorithms. The LSTM model underwent training for 50 epochs, while the metaheuristic algorithms were trained for 500 iterations. A population size of 100 was employed for all optimization algorithms examined in the study. Over 90% of the datasets were allocated for training the LSTM model, with the remaining portion utilized for training validation.

Table 2 Model definition for twenty (20) basic classical benchmark functions

ID	Function name	Description
F1	Wavy	$f(x) = \sum_{i=1}^n x_i^2 + (\sum_{i=1}^n 0.5ix_i)^2 + (\sum_{i=1}^n 0.5ix_i)^4$
F2	Zakharov	$f(x) = \frac{1}{n} \sum_{i=1}^n 1 - \cos(10x_i) e^{-\frac{1}{2}x_i^2}$
F3	Salomon	$f_{19}(x) = 1 - \cos\left(2\pi\sqrt{\sum_{i=1}^n x_i^2}\right) + 0.1\sqrt{\sum_{i=1}^n x_i^2}$
F4	Ackley	$f(x) = -20e^{(-0.2\sqrt{\frac{1}{n}\sum_{i=1}^n x_i^2})} - e^{(\frac{1}{n}\sum_{i=1}^n \cos(2\pi x_i))} + 20 + e^{(1)}$
F5	Alpine	$f(x) = \sum_{i=1}^n x_i \sin(x_i) + 0.1x_i $
F6	Discus	$f(x) = (x_1 - 1)^2 + \sum_{i=2}^n i(2x_i^2 - x_{i-1})^2$
F7	Griewank	$f(x) = 1 + \sum_{i=1}^n \frac{x_i^2}{1400} - \prod_{i=1}^n \cos(\frac{x_i}{\sqrt{i}})$
F8	HGBat	$f_{23}(x) = \left \left(\sum_{i=1}^D x_i^2 \right)^2 - \left(\sum_{i=1}^D x_i \right)^2 \right ^{1/2} + \left(0.5 \sum_{i=1}^D x_i^2 + \sum_{i=1}^D x_i \right) / D + 0.5$
F9	High Conditioned Elliptic	$f_{23}(x) = \sum_{i=1}^D (10^6)^{\frac{i-1}{D-1}} x_i^2$
F10	Brown	$f(x) = \sum_{i=1}^{n-1} (x_i^2)^{(x_{i+1}^2+1)} + (x_{i+1}^2)^{(x_i^2+1)}$
F11	Bent Cigar	$f_{20}(x) = x_1^2 + 10^6 \sum_{i=2}^D x_i^2$
F12	Levy	$f_{12}(x) = \sum_{i=1}^n (x_i - 1)^2 [\sin^2(3\pi x_{i+1})] + \sin^2(3\pi x_1) + x_n - 1 [1 + \sin^2(3\pi x_n)]$
F13	Powell	$f(x) = (x_1 + 10x_2)^2 + 5(x_3 + x_4)^2 + (x_2 - 2x_3)^4 + 10(x_1 - x_4)^4$
F14	Rastrigin	$f_9(x) = \sum_{i=1}^n [x_i^2 - 10\cos(2\pi x_i) + 10]$
F15	Rosenbrock	$f(x) = \sum_{i=1}^{n-1} [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2]$
F16	Sphere	$f_1(x) = \sum_{i=1}^n x_i^2$
F17	Sum of Squares	$f(x) = \sum_{i=1}^n ix_i^2$
F18	Sum-power	$f_8(x) = \sum_{i=1}^n x_i ^2$
F19	Sum of different power	$f_{21}(x) = \sum_{i=1}^d x_i ^{i+1}$
F20	Weierstrass	$f(x) = \sum_{i=1}^D (\sum_{i=0}^{20} [0.5^k \cos(2\pi \cdot 3^k (x_i + 0.5))])$

Table 3 A listing and model definition for ten (10) hybrid classical benchmark functions

ID	Function name	Definition
H1	Penalized	$f(x) = \frac{\pi}{n} X \left\{ 10 \sin^2(\pi y_i) + \sum_{i=1}^{n-1} (y_i - 1)^2 [1 + 10 \sin^2(\pi y_{i+1})] + (y_n - 1)^2 \right\} + \sum_{i=1}^n u(x_i, a, k, m)$ Where $y_i = 1 + \frac{1}{4}(x_i + 1)$, $u(x_i, a, k, m) = \begin{cases} k(x_i - a)^m & \text{if } x_i > a \\ 0 & \text{if } -a \leq x_i \leq a \\ k(-x_i - a)^m & \text{if } x_i < -a \end{cases}$ $a = 10, k = 100, m = 4$
H2	Composition 1	[F15, F9, F14]
H3	Composition 2	[F4, F9, F10, F14]
H4	Composition 3	[F2, F15, F14]
H5	Composition 4	[F9, F4, F14, F8, F6]
H6	SR-1	Shifted and rotated F11
H7	SR-2	Shifted and rotated F19
H8	SR-3	Shifted and rotated F2
H9	SR-4	Shifted and rotated F15
H10	SR-5	Shifted and rotated F14

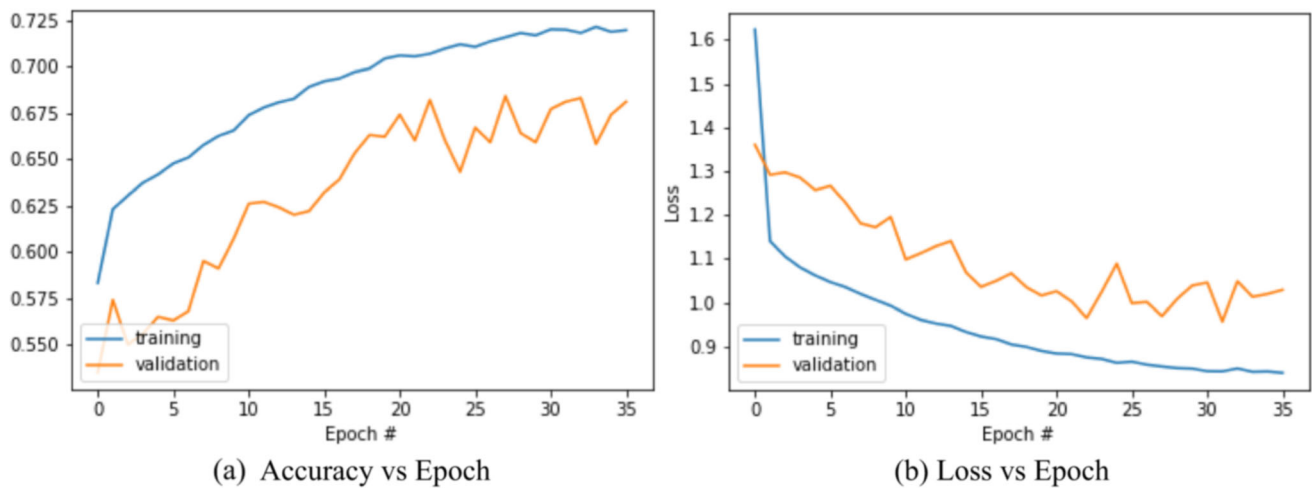


Fig. 6 Graph of the **a** accuracy and **b** loss function curves for training and validation of the LSTM deep learning model for biology-based algorithms using the EOSA algorithm as a benchmark

This listing of the parameter values allows for the reproducibility of the experimentation described in this section.

4.3 Dataset

The dynamic data, generated and structured for training the deep learning model, underwent additional preprocessing to serve as input for the model. To visualize the data, a time series approach was employed, as depicted in Fig. 5. For each optimization algorithm, a buffered stream of strings was collected by concatenating the elite solutions and current solution for the last few iterations. These buffered strings were then preprocessed and used as input data for the LSTM model. Figure 5 provides a representation of samples from the processed data.

The figure showcases a time series data representation approach with the first 15 rows of the processed data. Each plot represents a row of data, and their corresponding curves are labeled with legends x_1 and y , respectively. The figure provides a visual representation of the wavy curve for only the first few samples, out of the total 37,800 data samples used to train the evolution-based, swarm-based, and biology-based models.

4.4 Benchmark functions for performance evaluation

In this study, benchmark functions were employed to conduct experiments for both the deep learning-guided metaheuristic algorithm performance enhancement and the classical optimization process. The evaluation of all algorithms considered in this study was performed using

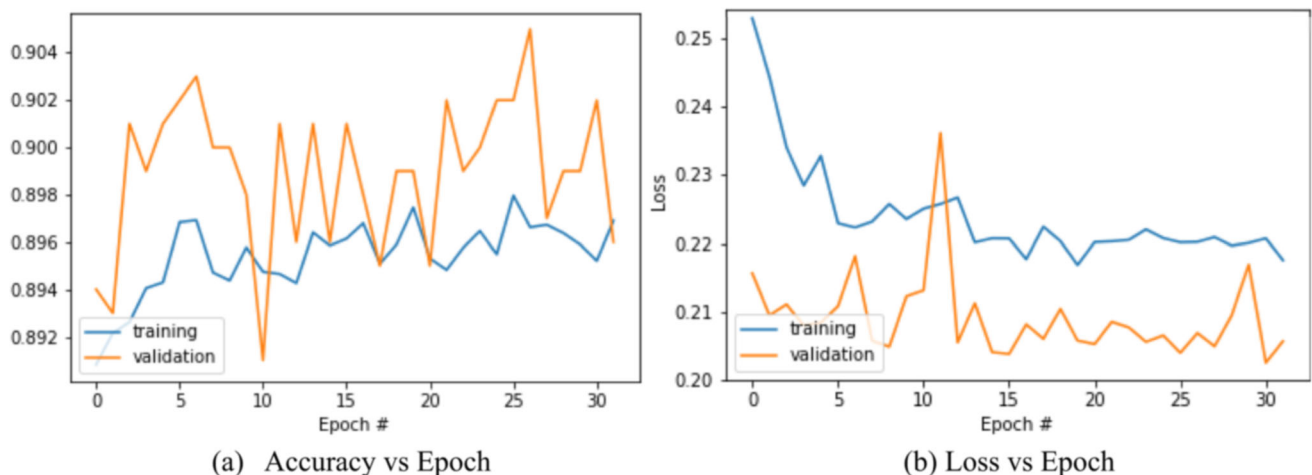


Fig. 7 Graph of the **a** accuracy and **b** loss function curves for training and validation of the LSTM deep learning model for evolution-based algorithms using the GA algorithm as a benchmark

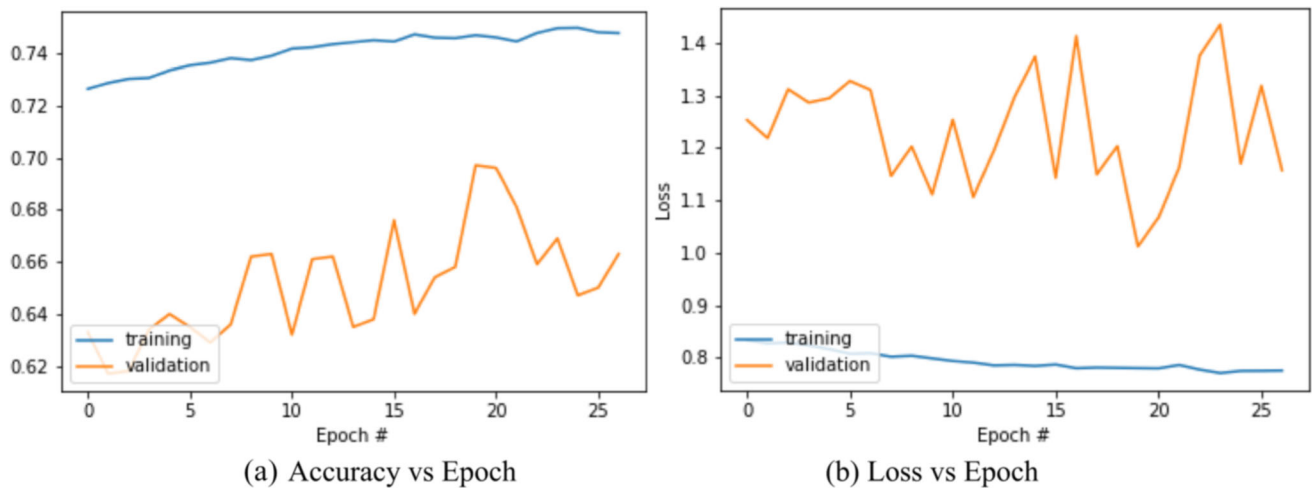


Fig. 8 Graph of the **a** accuracy and **b** loss function curves for training and validation of the LSTM deep learning model for swarm-based algorithms using the PSO algorithm as a benchmark

Table 4 A comparative analysis of the best solutions obtained for the non-complex benchmark functions for the biology-based methods, namely EOSA, IEOSA, SMA, VCS, and IWO

ID	EOSA	EOSA-DL	IEOSA	IEOSA-DL	IWO	IWO-DL	SMA	SMA-DL	VCS	VCS-DL
F1	6.498482	5.758636	1.77E+18	6.447791	8159.898	7620.679	4.238694	4.238694	7.9757	32.68712
F2	0	0	0.282851	0	0.116825	0.080365	0	0	6.47E−07	5.22E−07
F3	0	0	2136.127	1.04E−222	175.6057	235.4796	0	0	4.26E−06	1.78E−07
F4	4.44E−16	0	19.96677	19.96677	15.44839	15.01028	4.44E−16	4.44E−16	0.007331	0.008543
F5	0	10	15.44789	12.66324	2.320775	1.934939	0	0	0.00048	0.00068
F6	0	17.45187	85,609,639	2.27E−216	21,242.79	28,627.23	0	0	0.000281	0.000589
F7	0	0.767955	169.4167	0	34.07261	26.26677	0	0	2.87E−06	4.18E−06
F8	0.5	0	565.6588	0.5	1069.794	2990.329	0.306757	0.122996	0.436311	0.39927
F9	0	0	2E+08	3.38E−207	918,903.4	1,223,355	0	0	0.013087	0.003204
F10	10	0	10	10	10.00002	10.00002	10	10	10.00001	10.0001
F11	0	0	4.82E+09	2.95E+08	4.66E+08	5.22E+08	0	0	3.906978	2.571508
F12	3.036499	0.333014	0.214338	0.000898	3.331421	0.169384	1.79E−07	4.51E−06	0.47474	0.435687
F13	0	0	1.44E−233	1.74E−222	1.75E−06	5.68E−07	0	0	1.22E−05	1.55E−07
F14	0	0	46.78672	28.32943	50.31824	0	0	0	0.011663	0.007184
F15	3.920999	3.91771	3.852811	3.597666	222,387.7	64,805.12	0.54234		3.045575	3.178286
F16	0	0	25,215.07	8.35E−226	1737.493	2636.061	0	0	2.01E−05	2.62E−05
F17	0	0.804173	312.1785	2.66E−230	36.19454	35.20226	0	0	8.16E−05	5.35E−05
F18	1.69E−08	4.44E−16	2.91E−215	1.72E−231	0.226827	0.049226	0	0	6.63E−06	2.60E−06
F19	0	0	289,694.9	2.43E−112	17,529.34	18,508.46	0	0	6.16E−05	8.93E−06
F20	0	0	7.165134	0	2.027791	3.334294	0	0	0.001272	0.001102
Superior	4	7	1	17	9	10	1	2	7	13
Inferior	7	4	17	1	10	9	2	1	13	7
Compete	9		2		1		17		0	
Overall	7		17		10		2		13	

benchmark functions categorized into two categories: basic and hybrid benchmark functions. Table 2 provides an overview of the basic benchmark functions, which were

subsequently hybridized to obtain the complex functions defined in the second category. These benchmark functions

served as the foundation for assessing the performance of the various algorithms presented in this study.

Table 3 presents a compilation of derivable functions utilizing predefined functions. These complex functions were specifically chosen to illustrate the proposed method's practicability and to test its effectiveness. Among these functions, four were highly compositional, constructed by combining basic functions mentioned earlier in Table 2, while five were derived by applying shift rotation operations to certain basic functions. Only one of these hybrid functions exhibited significant complexity due to the inclusion of a penalization model.

The performance evaluation of the metaheuristic algorithms considered in this study utilized all the functions categorized as basic and hybrid, as outlined in Tables 2 and 3. For the experimentation involving the deep learning-guided initial solution composition and search process operation, the performance of the algorithms without the deep learning method was assessed using these functions. In the subsequent section, we provide a comprehensive breakdown of the results and engage in a detailed discussion of the findings from the study.

5 Results and discussion

The results and comparisons presented in this section focus on the performance of the LSTM model and its application to various metaheuristic algorithms, including EOSA, IEOSA, VCS, IWO, SMA, DE, GA, PSO, FFA, AO, HGS, SSA, and WOA, which were tested using the benchmark

functions described in Tables 2 and 3. The evaluation of the DL-based method is based on the analysis of classical benchmark functions. This section concludes with a discussion of the study's findings and the implications of incorporating deep learning into the design and performance of metaheuristic algorithms. Furthermore, this section also offers insights into the limitations of the study. To conduct comprehensive experimentation and refine our approach, we employed a set of 30 widely recognized functions, encompassing both complex and constrained benchmark functions. These functions are detailed in Tables 2 and 3, serving as the foundation for our extensive experimentation and approach refinement for the current deep learning-based optimization approach.

Figure 6 illustrates the trajectory of accuracy and loss values obtained from training a model for all biology-based metaheuristic algorithms. The training and validation curves are plotted to demonstrate the model's ability to learn the classification problem. The highest accuracy achieved during the 50 epochs for the biology-based algorithms was approximately 0.72 for training and 0.675 for validation. The trained model progressively learned the classification problem as the epoch number increased. Similarly, the loss values steadily decreased from 1.6 to below 0.9 as the training approached the defined number of epochs. Subsequently, this trained model was applied to incorporate deep learning support into the EOSA, VCS, IWO, and SMA optimization algorithms.

Figure 7 depicts the training algorithms for the evolution-based category. Notably, we observed distinct performance characteristics compared to the biology-based

Table 5 A comparative analysis of the best solutions for the complex benchmark functions for the biology-based methods, namely EOSA, IEOSA, SMA, VCS, and IWO

ID	EOSA	EOSA-DL	IEOSA	IEOSA-DL	IWO	IWO-DL	SMA	SMA-DL	VCS	VCS-DL
H1	0.137289	0.134846	1,031,081	0.218218	16,520.89	757.1523	2.62E−08	6.48E−07	0.160633	0.166403
H2	0	0.25795	12,921.29	19,089.5	965.9135	417.6018	0	0	0.001172	0.000838
H3	0.702385	0	1.10E+16	0.501816	1.91E+09	4.1E+08	0.048559	0.001417	0.108748	0.080821
H4	0	0	0.31394	0	0.251327	0.242786	0	0	6.71E−07	1.77E−06
H5	0	4.5	523,969.3	6.80E−211	695,574.1	191,633.5	0	0	0.005735	0.009161
H6	765,345.5	300,211	1.93E+09	26,669.18	1.58E+09	1.04E+09	200.5034	201.0021	397,533	344,355.5
H7	1200.001	1200.002	1813.288	1200	1200.865	1200.875	1200	1200	1200	1200
H8	1208.165	1204.438	11,456,818	1204.794	1211.302	1214.741	1204.239	1204.239	1214.502	1237.464
H9	400.4244	400.0934	6414.115	400.012	453.2738	489.137	400.0239	400.1044	400.1596	400.2731
H10	900.2196	900.4492	966.1869	900.1023	926.7677	936.9327	900	900	900.4227	900.3243
Superior	4	5	1	9	3	7	2	1	5	4
Inferior	5	4	9	1	7	3	1	2	4	5
Compete	1		0		0		7		1	
Overall	5		9		7		1		4	

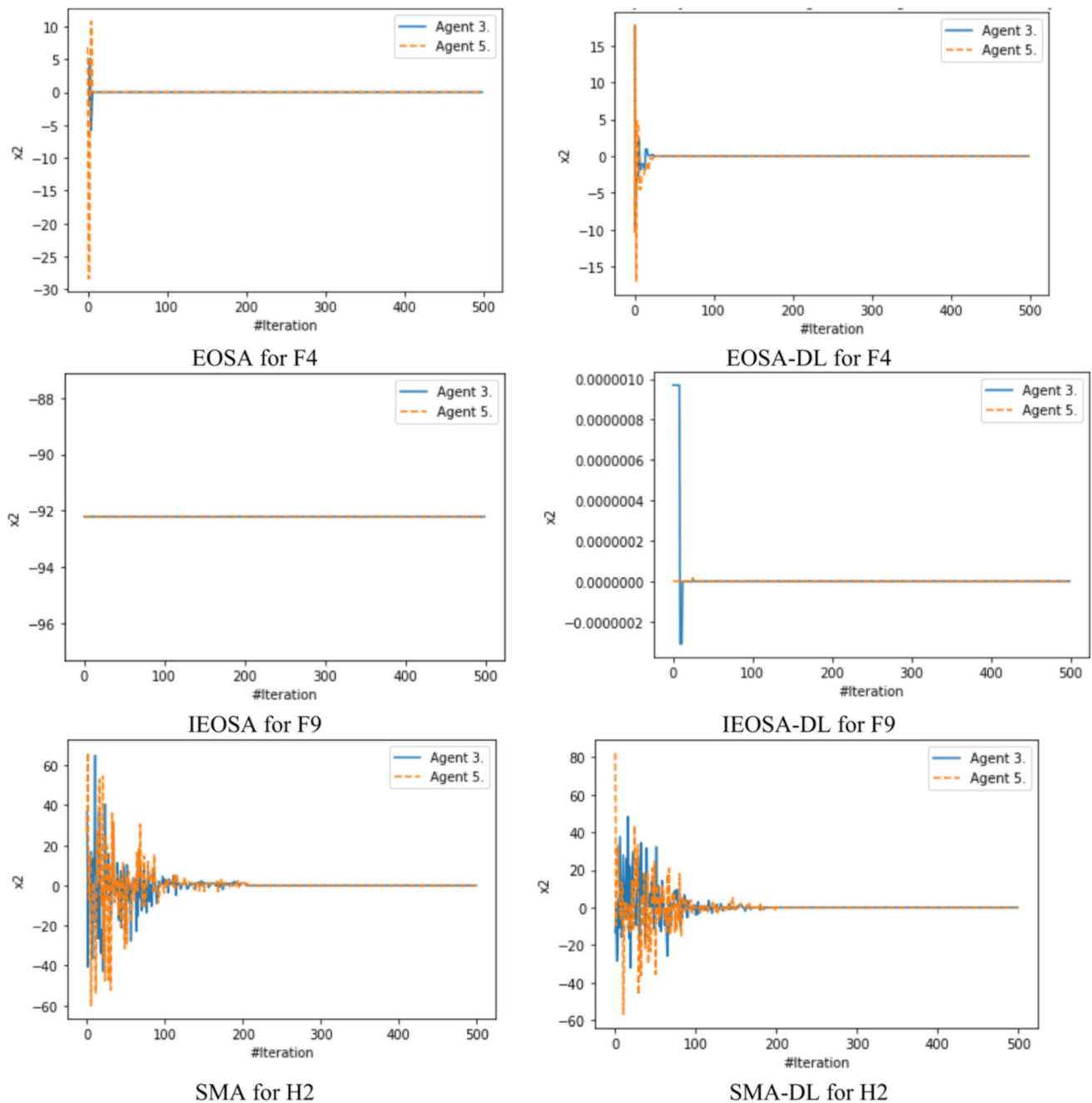


Fig. 9 Graph-based comparative analysis of biology-based optimization methods using solution trajectory for standard benchmark functions on EOSA, IEOSA, SMA, VCS and IWO

algorithms, emphasizing the importance of tailoring the neural network model training based on the categorization of metaheuristic algorithms. In this study, the evolution-based algorithms were trained using GA to enhance metaheuristic performance. Interestingly, the generalization pattern differed between the biology-based and evolution-based algorithms. The model training achieved a higher classification accuracy (0.90) for the evolution-based algorithms compared to the accuracy (0.72) observed

for the biology-based algorithms. However, the loss function curve for the biology-based method exhibited higher classification learning compared to the evolution-based method. It is worth mentioning that both GA and DE algorithms were considered in this study for the evolution-based approach.

The PSO algorithm was employed to train the LSTM model for swarm-based algorithms, including FFA, AO, HGS, SSA, and WOA. Figure 8 illustrates the results

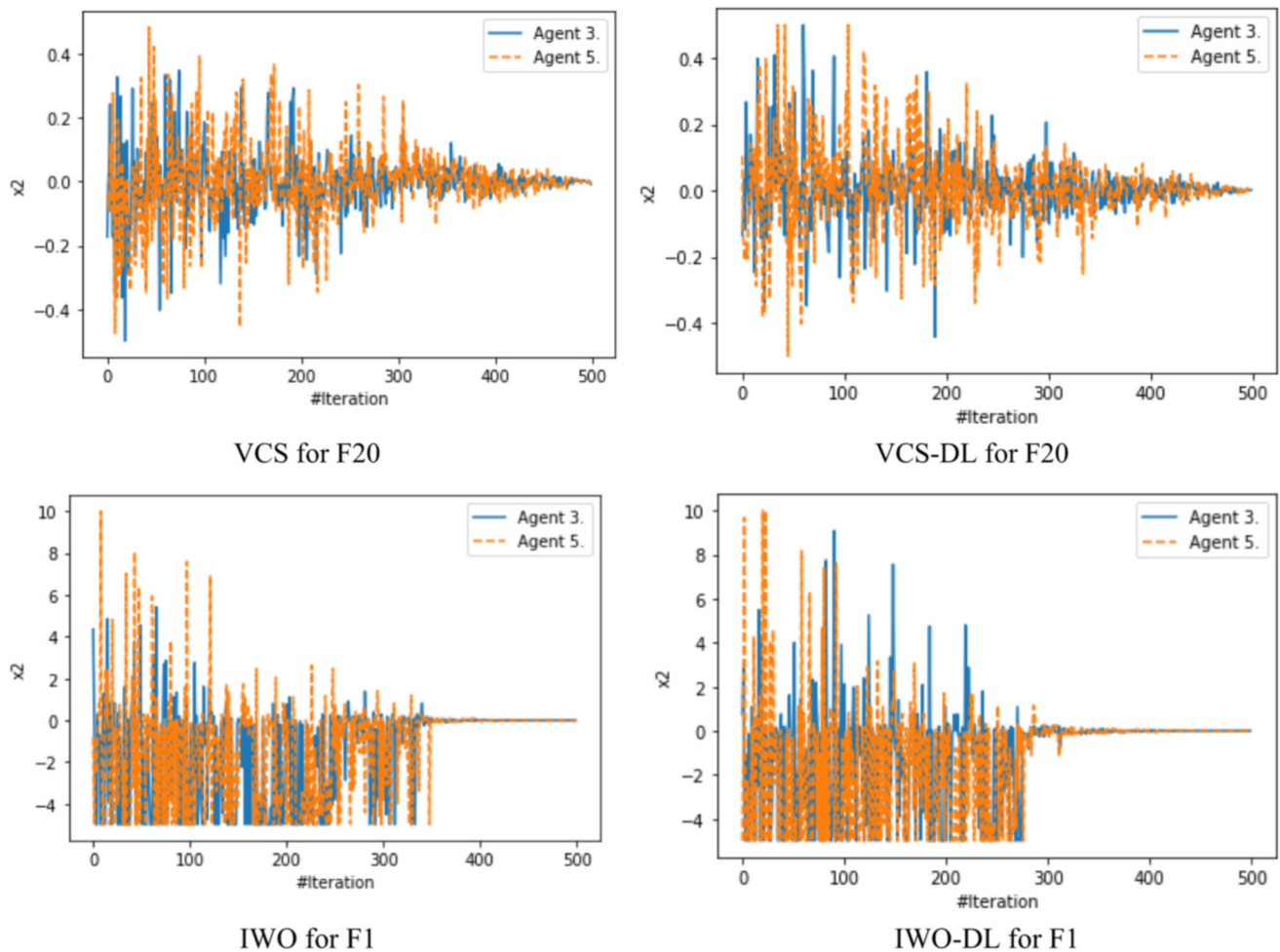


Fig. 9 continued

obtained from training and validating the LSTM model on algorithms within this category. Despite PSO's slower training compared to evolution and biology-based methods, it achieved competitive results. Notably, the highest classification accuracy achieved for training and validation of the model were 0.75 and 0.70, respectively, indicating superior performance. Furthermore, the loss function curve exhibited similarities to biology-based and evolution-based algorithms. The plot demonstrates that the swarm-based learning approach reached its lowest loss function value at 0.1, falling within the range observed in the two preceding categories.

Three LSTM models were utilized to enhance the performance of various optimization algorithms, including EOSA, IEOSA [22], SMA [36], VCS, IWO [38], GA, DE, AO, FFA, PSO, SSA, HGS, and WOA. In the subsequent paragraphs, we will provide a comparative analysis to evaluate the influence of integrating neural network models into optimization algorithms compared to training the metaheuristic algorithms using traditional approaches. As mentioned earlier, a category-based method was employed

to present the performance comparison, aiming to showcase the effectiveness of the trained models.

The analysis compared biology-based algorithms with their respective deep learning-enabled variants. Table 4 presents the results for EOSA, IEOSA, SMA, VCS, and IWO methods, alongside their DL-based counterparts (EOSA-DL, IEOSA-DL, SMA-DL, VCS-DL, and IWO-DL), for benchmark functions F1–F20. Each algorithm was evaluated against its DL-based variant to determine the superior performing option. In row 1, for example, EOSA-DL exhibited better performance than EOSA, IEOSA-DL outperformed IEOSA, IWO-DL surpassed IWO, while SMA and SMA-DL performed similarly. Notably, VCS showed better performance than VCS-DL. This comparative analysis is repeated for rows 2–20, covering all benchmark functions.

The summary of algorithm performance is presented in the last four rows. The 'Superior' row indicates that EOSA-DL outperformed EOSA in 7 functions, while EOSA outperformed EOSA-DL in only 4 functions, and there was a competitive performance in 9 functions. Similarly, IEOSA-

Table 6 A comparative analysis of the best solutions obtained for the non-complex benchmark functions for the evolution-based methods, namely GA and DE

ID	GA	GA-DL	DE	DE-DL
F1	13,688.47	14.77083	479.8773	4.595938
F2	0.000271	1.40E−17	0.045475	1.28E−11
F3	1.11076	6.04E−10	0.099995	6.04E−10
F4	3.251889	1.76E−06	0.057437	3.61E−05
F5	0.00322	7.79E−06	0.022673	7.28E−07
F6	64,063.15	6.04E−09	0.010943	5.10E−10
F7	0.463283	1.01E−09	0.146295	7.19E−10
F8	1.237795	0.500085	0.22878	0.27829
F9	19,869.98	5.83E−12	0.107528	4.88E−08
F10	10.00388	10	10.02315	10
F11	1,687,962	3.93E−09	126.4464	5.43E−06
F12	0.502025	0.086645	0.014393	7.00E−06
F13	0.000593	3.02E−08	4.02E−06	3.03E−08
F14	1.269816	7.62E−13	3.732614	1.89E−07
F15	33.38654	3.990104	0.168895	0.096121
F16	1.212519	6.04E−09	8.69E−05	1.89E−09
F17	0.020781	1.89E−15	1.39E−06	3.04E−11
F18	0.000119	6.04E−09	2.85E−08	5.99E−11
F19	0.075423	4.70E−13	0.000295	9.02E−15
F20	0.002936	6.56E−07	0.006351	2.68E−08
Superior	0	20	1	19
Inferior	20	0	19	1
Compete	0		0	
Overall	20		19	

DL demonstrated superior performance in 17 functions compared to IEOSA, which outperformed in only 1 function. IWO-DL and VCS-DL showcased significant superiority over their respective IWO and VCS variants, with 10 and 13 benchmark functions, respectively. On the other hand, IWO and VCS outperformed their DL variants in only 9 and 7 functions, respectively.

Interestingly, SMA did not exhibit significant performance improvement with the DL method. SMA-DL performed better than SMA in 2 functions, while SMA outperformed SMA-DL in only 1 function. Comparing the impact of the DL-based approach across all algorithms, the overall performance indicated that the approach proved better than EOSA, IEOSA, IWO, SMA, and VCS in 7, 17, 10, 2, and 13 benchmark functions, respectively. These findings highlight the effectiveness of DL-based enhancements in improving the performance of various algorithms. However, it is important to note that the impact of the DL method varied across different algorithms, with some

Table 7 A comparative analysis of the best solutions obtained for the complex benchmark functions for the evolution-based methods, namely GA and DE

ID	GA	GA-DL	DE	DE-DL
H1	0.003945	0.030676	0.000932	4.33E−05
H2	27.01671	3.43E−08	2.561938	2.37E−08
H3	23.38173	0.440572	0.471918	0.002219
H4	0.152721	2.65E−16	0.126487	1.38E−11
H5	641.3719	3.37E−13	0.22802	2.19E−07
H6	1,221,422	20,193.73	262.3003	200.4145
H7	1200.005	1200.004	1200	1200
H8	1204.397	1204.986	1204.239	1362.492
H9	403.3881	400.1053	400.0422	400.0135
H10	900.2922	900.0304	900.2071	900
Superior	1	9	2	7
Inferior	9	1	7	2
Compete	0		1	
Overall	9		7	

algorithms exhibiting more significant improvements than others.

The computational relevance of the DL-based method was thoroughly investigated for hybrid and complex benchmark functions. The results, as presented in Table 5, demonstrate the robust performance of the proposed method, even in the presence of complex benchmark functions. For example, in row 1 of the table, it is observed that EOSA-DL outperforms EOSA, which is consistent with the similar performance observed between IEOSA-DL and IEOSA, as well as IWO-DL and IWO. However, there is a reverse performance for SMA-DL, SMA, VCS-DL, and VCS, where the DL-enhanced algorithms underperform compared to their base methods.

A comprehensive summary of the results reveals that EOSA-DL, IEOSA-DL, and IWO-DL outperform their base methods in 5, 9, and 7 benchmark functions, respectively, out of the 10 analyzed complex functions. Conversely, SMA-DL and VCS-DL are outperformed by their base methods in 2 and 5 benchmark functions, respectively. Furthermore, specific comparisons between algorithms show that EOSA and EOSA-DL yield identical results for one benchmark function, SMA and SMA-DL yield the same results for seven benchmark functions, and VCS and VCS-DL yield the same results for one benchmark function.

In terms of overall performance, IEOSA-DL and IWO-DL demonstrate a significant impact through the utilization of DL methods, achieving values of 9 and 7, respectively. This is followed by values of 5, 4, and 1 for EOSA-DL, SMA-DL, and VCS-DL, respectively. These findings

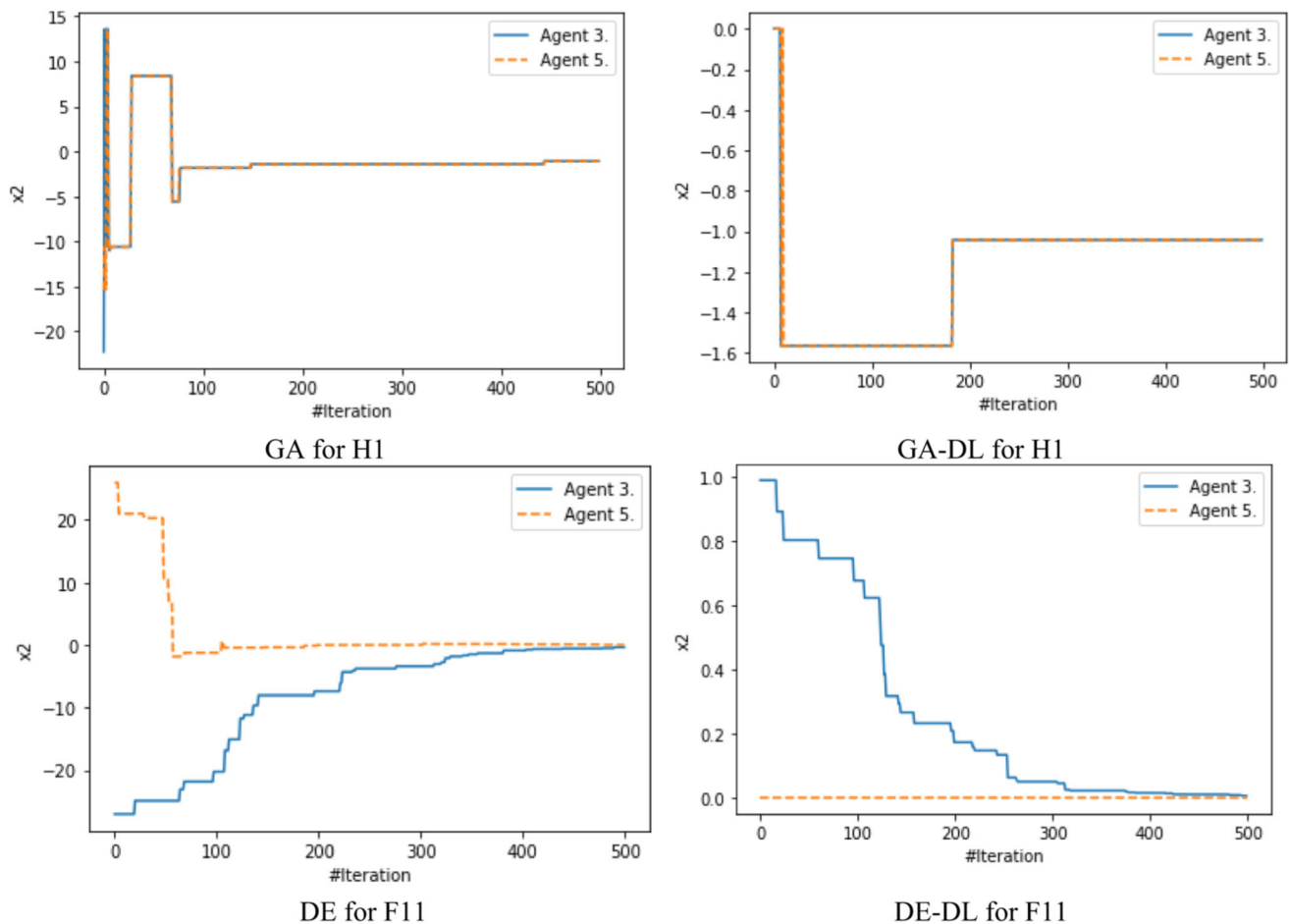


Fig. 10 Graph-based comparative analysis of evolution-based optimization methods using solution trajectory for standard benchmark functions on GA and DE

highlight the effectiveness of DL-based methods, particularly IEOSA-DL and IWO-DL, in enhancing the performance of the algorithms on complex benchmark functions. However, it should be noted that SMA-DL and VCS-DL may not provide significant improvements over their base methods in certain cases. The constraints, particularly concerning the subpar performance of DL-based approaches in some instances, can be ascribed to the inadequacy of the datasets generated and employed in training these models.

Figure 9 showcases the trajectory of solutions computed over 500 iterations for various biology-based algorithms and their DL-enhanced counterparts. Specifically, it includes EOSA and EOSA-DL for the F4 function, IEOSA and IEOSA-DL for F9, SMA and SMA-DL for H2, VCS and VCS-DL for F20, and IWO and IWO-DL for F1. The trajectories represent selected individuals (agents) in the solution space. The results demonstrate notable improvements in the DL-enhanced curves compared to their base methods. For instance, EOSA-DL exhibits a smoother convergence within the acceptable range of $[-15, 15]$ for

the F4 function, while EOSA converges within the range of $[-30, 10]$. Similarly, IEOSA-DL, SMA-DL, VCS-DL, and IWO-DL demonstrate outstanding performances with narrower ranges of $[-0.0000002, 0.0000010]$, $[-60, 80]$, and $[-0.4, 0.4]$ respectively, compared to the wider ranges of $[-96, 88]$, $[-60, 60]$, $[-0.4, 0.4]$, and $[-4, 10]$ for IEOSA, SMA, VCS, and IWO on their respective benchmark functions.

In some cases, the DL-based methods exhibit faster convergence in earlier iterations compared to their base methods, which converge at later iterations. These findings suggest that the utilization of DL methods has been successful in improving the performance of the biology-based algorithms. In general, the results demonstrate the effectiveness of DL-enhanced approaches in achieving smoother convergence, narrower solution ranges, and in some cases, faster convergence compared to their base methods. These findings highlight the successful integration of DL techniques to enhance biology-based optimization algorithms.

Table 6 presents a comparative analysis of evolution-based algorithms on non-complex benchmark functions.

Table 8 A comparative analysis of the best solutions obtained for the non-complex benchmark functions for the swarm-based methods, namely AO, FFA, PSO, SSA, HGS, and WOA

ID	AO	AO-DL	FFA	FFA-DL	HGS	HGS-DL	PSO	PSO-DL	SSA	SSA-DL	WOA	WOA-DL
F1	4.238832	4.238726	23.24297	4.239828	4.28352	5.047116	12,711.72	8.85019	4.238694	4.238694	4372.612	6.674437
F2	0	0	0.089849	2.93E-14	0	0	0.125687	2.93E-14	1.40E-16	2.79E-17	0	0
F3	0	0	0.699878	9.31E-14	0	0	0.09991	9.31E-14	8.47E-44	1.77E-40	0.099873	8.99E-135
F4	6.48E-09	4.00E-15	2.93515	1.73E-06	4.44E-16	4.44E-16	2.330574	1.72E-06	1.89E-12	2.21E-13	2.81435	7.55E-15
F5	1.56E-28	3.47E-64	0.505786	1.01E-07	0	0	1.892758	1.01E-07	2.82E-07	1.00E-07	0.002529	5.08E-79
F6	0.052854	7.46E-11	7366.135	7.46E-11	0	0	30,000.3	7.46E-11	5.93E-29	2.88E-24	1.57E-117	1.09E-124
F7	0	0	15.64056	1.00E-13	0	0	0.24467	1.00E-13	1.89E-15	0	0.118211	0
F8	0.084566	0.165768	2.293964	0.094964	0.5	0.383339	0.340722	0.497555	0.000441	7.11E-11	0.142562	0.315085
F9	0.139322	1.61E-08	28,691.67	1.61E-08	0	0	168,489.3	1.61E-08	3.13E-24	7.21E-26	8.81E-109	1.55E-130
F10	10.00073	10	10.00429	10	10	10	14	10	10	10	10	10
F11	471.5924	6.67E-07	8,015.024	6.67E-07	0	0	10,000	6.67E-07	8.15E-35	1.33E-31	4.91E-116	1.39E-121
F12	0.001982	0.014956	0.319136	0.038651	0.034048	0.003139	1.284464	0.031453	6.75E-31	1.35E-31	3.83E-12	4.15E-12
F13	4.60E-09	0	0.037532	7.78E-17	0	0	5	7.78E-17	1.44E-22	5.36E-32	4.61E-06	8.07E-32
F14	3.131821	0	13.05533	1.85E-10	0	0	11.37371	1.85E-10	8.99E-13	1.90E-13	0	1.78E-15
F15	0.000346	0.000442	350.5791	1.594908	0.023528	0.716868	90,002.21	3.826559	0	0	3.107867	0
F16	0	0	35.44156	9.31E-13	0	0	8.04E-09	9.31E-13	3.16E-44	2.62E-43	5.75E-116	6.44E-127
F17	0	0	0.398423	2.61E-12	0	0	1.20E-10	6.60E-13	9.07E-37	7.62E-44	1.52E-115	8.53E-133
F18	0	0	0.000878	9.31E-13	0	0	6.64E-13	8.22E-14	1.88E-36	5.83E-43	3.93E-119	1.12E-131
F19	0.000477	1.52E-45	218.8635	8.58E-09	0	0	10,100	8.58E-09	4.65E-06	7.32E-17	1.32E-111	3.99E-131
F20	0	0	0.006486	1.01E-10	0	0	5.57E-10	7.97E-11	5.46E-13	1.78E-15	0	1.78E-15
Superior	3	10	0	20	2	2	1	19	4	13	4	14
Inferior	10	3	20	0	2	2	19	1	13	4	14	4
Compete	7	0	0	0	16	0	0	0	3	0	2	0
Overall	10	20	20	0	0	0	19	19	13	13	14	14

Table 9 A comparative analysis of the best solutions obtained for the complex benchmark functions for the swarm-based methods, namely AO, FFA, PSO, SSA, HGS, and WOA

ID	AO	AO-DL	FFA	FFA-DL	HGS	HGS-DL	PSO	PSO-DL	SSA	SSA-DL	WOA	WOA-DL
H1	4.70E-07	2.42E-07	0.671153	0.00039	0.000509	0.125592	2.82998	0.516659	1.73E-19	6.22E-22	1.01E-16	9.67E-18
H2	0.00016	0	67.98231	1.42E-14	0	0	5.683572	1.42E-14	1.17E-13	7.11E-15	9.94958	0
H3	2.94E-08	1.48E-07	5.299662	0.110897	0.024561	0.413265	225.5386	0.540798	0	1.11E-15	0.022468	0.138105
H4	0	0	0.251327	2.93E-14	0	0	0.218588	2.93E-14	1.40E-16	2.09E-17	0	0
H5	0.842201	1.61E-08	9386.14	1.61E-08	0	0	168.489.3	1.61E-08	1.73E-23	3.52E-37	2.56E-118	3.04E-125
H6	661.0858	247.3473	4,118,155	2777.89	221.8242	204.3099	461.2396	225,855.8	200	200	200	200
H7	1200	1200	1200.028	1200	1200	1200	1204.95	1200.008	1200	1200	1200	1200
H8	1204.241	1204.266	1204.456	1204.239	1204.348	1204.252	1204.239	1204.239	1204.241	1204.301	1204.239	1204.241
H9	404.0175	400	402.7173	400.0344	400	400.0312	401.1953	400.575	400	400	400	400
H10	902.9963	900.0004	908.7949	900.0017	900	900.0001	928.9384	900.7085	900	900	906.9647	900
Superior	2	6	0	10	4	2	1	8	1	5	2	4
Inferior	6	2	10	0	2	4	8	1	5	1	4	2
Compete	2		0		4		1		4		4	
Overall	6		10		2		8		5		4	

The comparison involves GA, GA-DL, and DE-DL methods, and it is observed that GA-DL and DE-DL outperform their corresponding base methods, as exemplified in row 1. A summary report of the performance analysis reveals that GA-DL and DE-DL demonstrate superiority over their base methods, GA and DE, in 20 and 19 benchmark functions, respectively. In contrast, GA achieves better results in 0 benchmark functions, while DE outperforms in only 1 benchmark function. Notably, there is no reported competition between the DL-based methods and the classical optimization methods.

The comparison results indicate that GA-DL outperforms its base method in all 20 benchmark functions, while DE-DL demonstrates the best performance in 19 out of the 20 benchmark functions. These findings highlight the effectiveness of GA-DL and DE-DL in optimizing the non-complex benchmark functions, surpassing the performance of their base methods. The DL-based approaches demonstrate superior performance and prove to be more effective in solving the optimization problems at hand.

Table 7 presents a comparison of the complex hybrid functions on GA and GA-DL, as well as DE and DE-DL algorithms. The results indicate that DE-DL outperforms the DE method, while GA demonstrates better performance compared to GA-DL, as shown in row 1. Analyzing the performance across all 10 benchmark functions, GA-DL and DE-DL exhibit superior performance over their corresponding base algorithms in 9 and 7 benchmark functions, respectively. In contrast, GA and DE demonstrate better performance in only 1 and 2 benchmark functions, respectively. There is no observed competitive performance between GA and GA-DL, while there is one case where DE and DE-DL exhibit competitive performance. Overall performance evaluations highlight that GA-DL benefits from the proposed method discussed in this study compared to DE-DL. This indicates that the method employed in GA-DL yields improved results, emphasizing its effectiveness in optimizing the complex benchmark functions compared to DE-DL.

Figure 10 illustrates the convergence trajectories of selected individuals (agents) in the population for biology-based algorithms. Specifically, it showcases the convergence curves of GA and GA-DL for the H1 benchmark function, as well as DE and DE-DL for the F11 benchmark function. Upon analysis, it becomes evident that the DL-improved methods exhibit smoother convergence and a more favorable range compared to classical optimization methods. For instance, GA-DL demonstrates convergence within the interval $[-1.6, 0.0]$, while GA converges within a wider range of $[-20.0, 15.0]$ for the H1 benchmark function. Similarly, DE-DL converges within the range of $[0.0, 1.0]$, whereas DE shows convergence within a broader range of $[-20.0, 20.0]$ for the F11 benchmark function.

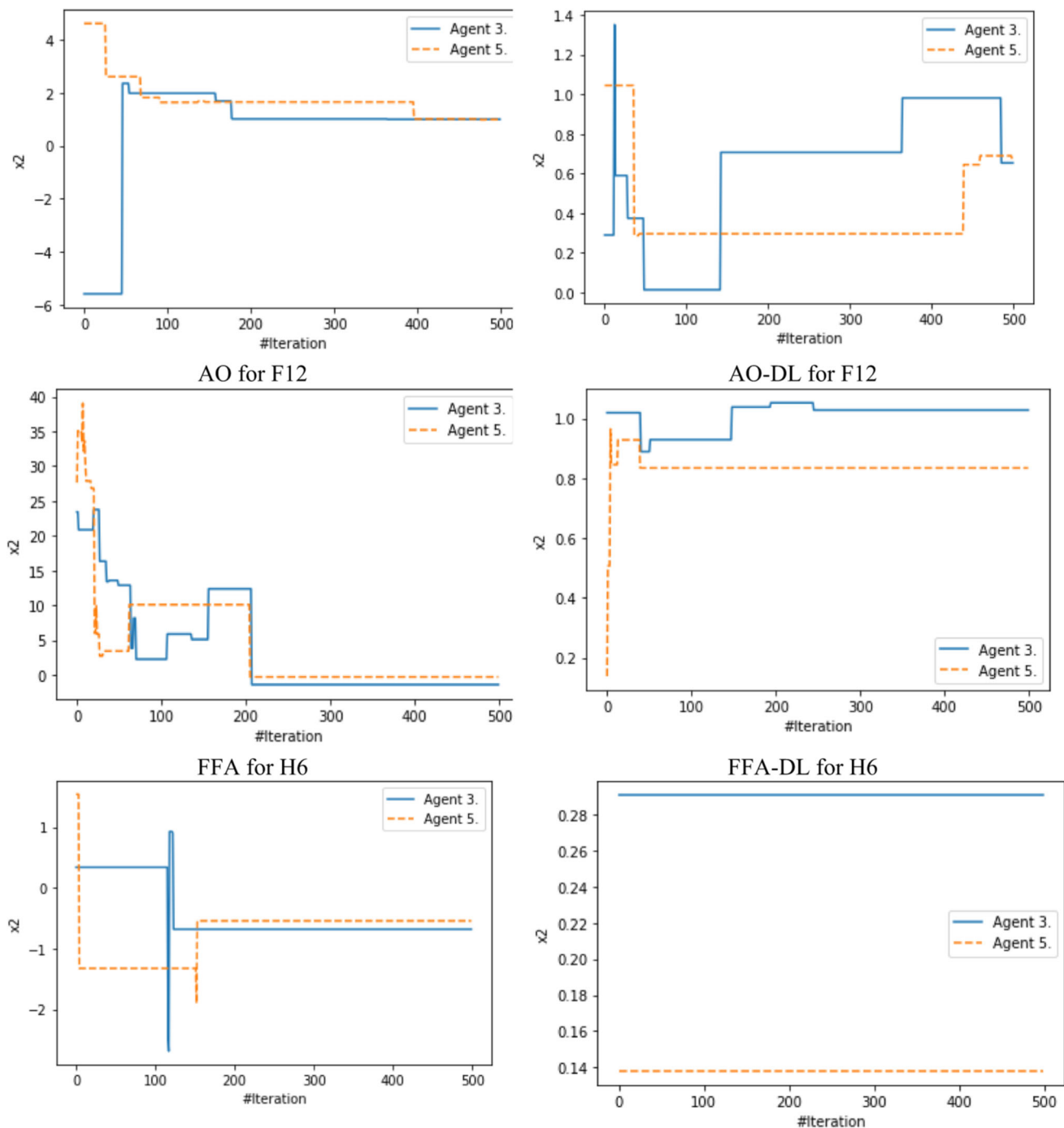


Fig. 11 Graph-based comparative analysis of swarm-based optimization methods using solution trajectory for standard benchmark functions on AO, FFA, PSO, SSA, HGS, and WOA

This comparative analysis highlights the significance of enhancing evolution-based metaheuristic methods through DL techniques. The DL-improved algorithms yield smoother convergence curves and more desirable convergence ranges, demonstrating their relevance and effectiveness in optimizing the benchmark functions.

Table 8 presents the results of swarm-based methods on non-complex functions. The comparison includes AO, FFA, PSO, SSA, HGS, and WOA with their corresponding DL-based methods: AO-DL, FFA-DL, PSO-DL, SSA-DL, HGS-DL, and WOA-DL. The findings reveal that AO-DL, FFA-DL, PSO-DL, SSA-DL, and WOA-DL outperformed their respective base methods in benchmark functions 10,

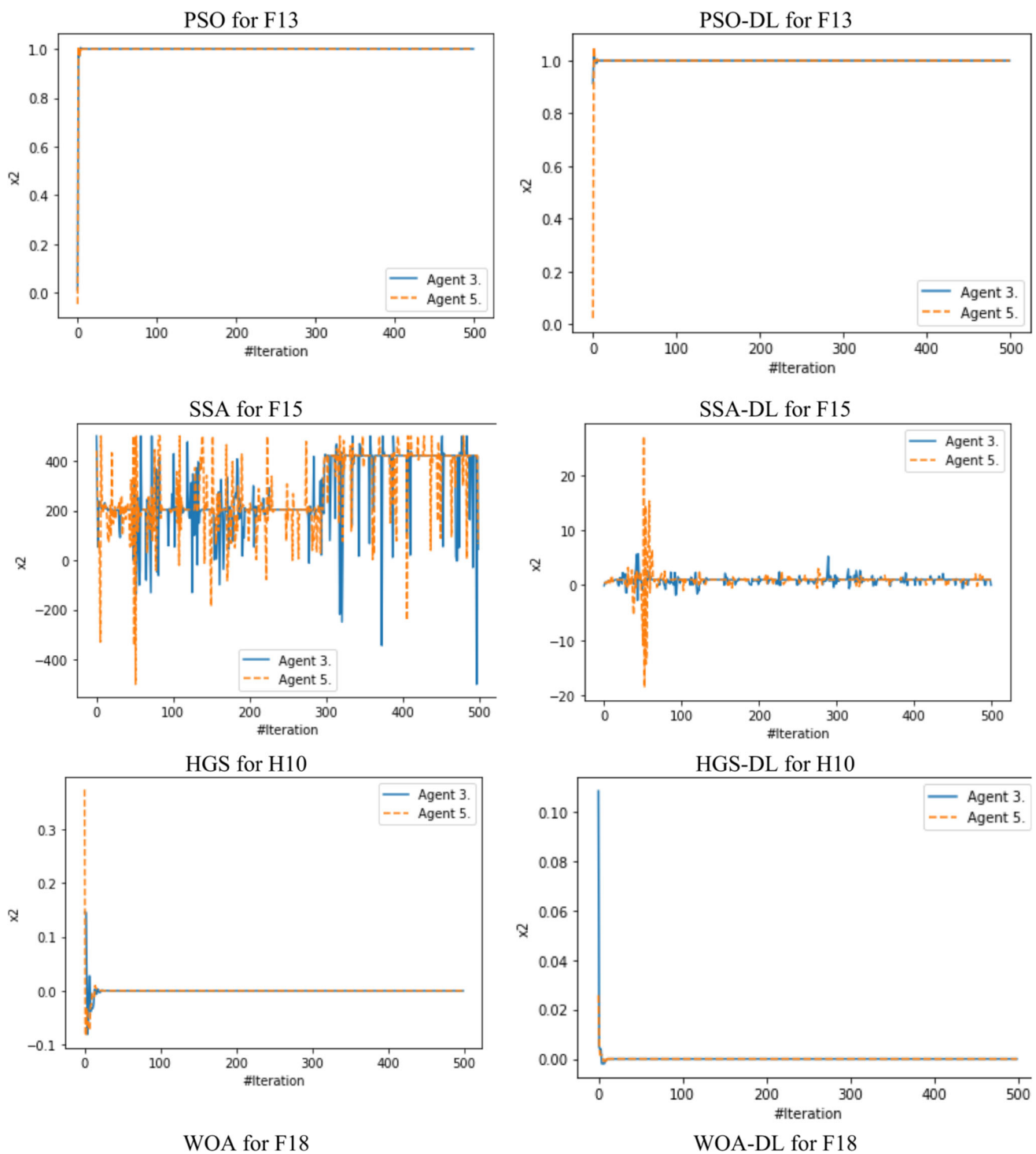


Fig. 11 continued

20, 19, 13, and 14. Notably, HGS and HGS-DL demonstrated strong competition, with 16 benchmark function evaluations resulting in similar performance, while each exhibiting superiority over the other in 2 benchmark function evaluations.

The overall performance evaluation highlights significant improvements achieved by FFA-DL and PSO-DL through the proposed knowledge-aware method discussed in this study. Although the training process for the PSO model was slower compared to other categories, the results remained competitive. These results underscore the

Table 10 A listing of twenty-five (25) selected CEC functions applied for evaluation of the based metaheuristic algorithms and their corresponding DL-based methods

ID	Function description	ID	Function description	ID	Function description
C1	S CEC01	C9	SR CEC08	C17	S [CEC09, CEC08, CEC01]
C2	S CEC02	C10	S CEC09	C18	S [CEC02, CEC12, CEC08]
C3	S CEC03	C11	SR CEC09	C19	S [CEC07, CEC06,CEC04, CEC14]
C4	S CEC04	C12	SR CEC10	C20	S [CEC12, CEC03,CEC13, CEC08]
C5	S CEC05	C13	SR CEC11	C21	S [CEC14, CEC12,CEC04, CEC09, CEC01]
C6	S CEC06	C14	SR CEC12	C22	S(1,2,3,4,5) [C04, C01,C02, C03, C01]
C7	S CEC07	C15	SR CEC13	C23	S(1,2,3) [C10, C09,C14]
C8	S CEC08	C16	SR CEC14	C24	S(1,2,3) [C11, C09,C01]
				C25	S(1,2,3,4,5) [C11,C13,C01,C06, C07]

Table 11 A comparative analysis of the best solutions for the IEEE CEC 2017 functions for the biology-based methods, namely EOSA, IEOSA, SMA, VCS, and IWO

ID	EOSA	EOSA-DL	IEOSA	IEOSA-DL	IWO	IWO-DL	SMA	SMA-DL	VCS	VCS-DL
C1	312.5571	359.8505	34,982,320	4388.172	636,502.1	743,163	100.000	100.0083	840.254	155.331
C2	157,371.3	791,435.5	1.41E+10	131,420.3	6.44E+08	1.77E+09	200.829	200.3685	196,545.5	673,515.8
C3	307.7134	301.7894	3,509,528	304.4326	22,735.62	31,477.98	300.000	300	306.872	302.432
C4	400.1094	400.0688	6414.115	400.3447	518.2482	429.445	400.015	400.0818	400.236	400.274
C5	503.5845	503.5897	501.7069	502.8703	520	520	500.000	500.0002	503.051	503.139
C6	600.0018	600.0005	602.9583	600.0017	603.5105	603.2198	600	600	600.002	600.002
C7	700.3335	700.0927	733.0892	700.1145	723.7965	703.4157	700.044	700.0443	700.058	700.059
C8	800.4958	800.866	841.4717	800.0486	837.6887	828.3384	800	800	800.410	800.539
C9	900.7646	901.1168	968.1392	900.0754	935.3755	938.0338	900	900	900.57	900.436
C10	1014.55	1003.205	3090.873	1037.634	1924.075	1971.681	1000.13	1000	1008.65	1007.96
C11	1108.131	1107.974	2914.299	1104.032	2120.137	1822.723	1100.25	1107.079	1107.73	1113.82
C12	1202.732	1201.42	1214.751	1201.342	1200.763	1200.6	1200.003	1200.009	1209.385	1205.272
C13	1301.244	1301.78	1301.222	1300.563	1301.375	1301.605	1300.243	1300.237	1301.869	1301.875
C14	1400.505	1400.501	1433.896	1400.586	1408.237	1407.119	1400.227	1400.025	1400.569	1400.545
C15	1500.058	1500.954	1,436,971	1500	1500.606	1523.128	1500.012	1500	1500.029	1500.035
C16	1600.683	1601.36	1602.243	1600.829	1600.958	1600.899	1600	1600	1601.313	1601.586
C17	1717.718	1772.815	44,909,579	1846.95	23,070.73	16,795.27	1700.001	1700	1715.047	1711.89
C18	1804	1804	3224.574	1802.026	5639.937	4742.654	1800	1800	1803.718	1803.862
C19	1900.485	1900.517	1903.658	1900.808	1901	1901	1900	1900.165	1900.286	1900.299
C20	2305.275	2032.346	4.81E+09	2005.73	21,896.27	13,166.08	2000.998	2000.995	2015.334	2009.437
C21	2102.14	2102.216	56,197.56	2101.209	6767.212	9223.385	2100	2100	2101.028	2101.577
C22	2398.888	2396.427	3090.709	2345.341	2606.087	2598.948	2300.242	2300.242	2386.497	2387.924
C23	2436.428	2452.2	2596.508	2424.727	2787.195	2839.441	2400.003	2400.004	2451.035	2451.879
C24	2546.721	2533.762	2689.335	2524.516	2681.214	2681.525	2500.016	2500.005	2546.242	2539.812
C25	2667.236	2674.63	3007.953	2703.18	2879.134	2881.486	2600.005	2600.008	2688.952	2675.19
Superior	13	11	1	24	10	13	8	9	14	10
Inferior	11	13	24	1	13	10	9	8	10	14
Compete	1		0		2		8		1	
Overall	13		24		13		9		14	

Table 12 A comparative analysis of the best solutions obtained for the IEEE CEC 2017 functions for the evolution-based methods, namely GA and DE

ID	GA	GA-DL	DE	DE-DL
C1	1499.328	1605.224	100.0129	100.0023
C2	341,042.4	25,715.9	235.1056	200.1609
C3	43,999.6	6700.069	300.0412	300.0003
C4	403.103	400.1634	400.006	400.0149
C5	502.8241	501.8392	502.7203	500.0883
C6	600.003	600.0007	600.0002	600
C7	700.4788	700.2563	700.1368	700.021
C8	800.8196	800.062	803.4688	800
C9	900.8248	900.0442	902.041	900
C10	1002.482	1000.832	1003.107	1000
C11	1103.413	1101.048	1103.351	1100
C12	1200.047	1200.103	1200.206	1200.003
C13	1300.624	1300.637	1300.159	1300.209
C14	1400.387	1400.4	1400.104	1400.479
C15	1500.172	1500.061	1500.008	1500
C16	1600.424	1600.419	1600.331	1600.749
C17	7952.204	1936.824	1700.421	1700.01
C18	1805.52	1802.713	1800.481	1800.602
C19	1900.199	1900.17	1900.151	1900.002
C20	3,052,015	2101.597	2000.93	2000.143
C21	2144.41	2100.655	2100.064	2100
C22	2408.242	2337.29	2307.466	2300.556
C23	2469.498	2435.899	2522.083	2405.266
C24	2569.841	2531.23	2649.252	2504.568
C25	2735.085	2650.008	2653.642	2600.225
Superior	4	21	5	20
Inferior	21	4	20	5
Compete	0		0	
Overall	21		20	

effectiveness of the DL-based methods, particularly FFA-DL and PSO-DL, in enhancing the performance of swarm-based algorithms on non-complex benchmark functions.

The evaluation of hybrid and complex benchmark functions was carried out, comparing the performance of base methods with their corresponding DL-based methods. Table 9 presents the results obtained for these 10 functions, involving AO, FFA, PSO, SSA, HGS, and WOA algorithms, as well as their respective DL-based hybrids: AO-DL, FFA-DL, PSO-DL, SSA-DL, HGS-DL, and WOA-DL. When analyzing the results of the complex benchmark functions, it is observed that the HGS algorithm competes strongly with the hybrid method, as HGS-DL outperforms HGS in only 2 functions, while HGS demonstrates superiority in 4 functions over HGS-DL. On the other hand,

AO-DL, FFA-DL, PSO-DL, SSA-DL, and WOA-DL exhibit impressive performance, with 6, 10, 8, 5, and 4 benchmark function evaluations, respectively, surpassing their base counterparts: AO, FFA, PSO, SSA, and WOA, which achieved 2, 0, 1, 1, and 2 evaluations, respectively, compared to their hybrid methods.

In terms of competitiveness, AO, HGS, PSO, SSA, and WOA achieved benchmark function evaluations of 2, 4, 1, 4, and 4, respectively. These results indicate a close competition between the base algorithms and their corresponding DL-based methods. Overall, the performance evaluation indicates that FFA benefited the most, achieving a value of 10, followed by PSO with a value of 8, and AO with a value of 6. These findings highlight the effectiveness of the DL-based hybrid methods, particularly FFA-DL and PSO-DL, in improving the optimization performance on the complex benchmark functions.

Figure 11 presents a comparison of the performance between AO and AO-DL using the F12 benchmark function, FFA and FFA-DL using H6, PSO and PSO-DL using F13, SSA and SSA-DL using F15, HGS and HGS-DL using H10, and WOA and WOA-DL using F18. The convergence curves for selected individuals in the population were utilized to generate the plots in the figure. Upon analysis, several observations can be made. Firstly, for the H6 benchmark function, the convergence curve of FFA-DL demonstrates superior performance over FFA, particularly within the range of [0.2, 1.0]. This indicates that the hybrid method shows improved convergence for this specific benchmark function. Secondly, in the case of the H10 benchmark function, the convergence curve of HGS-DL exhibits better performance compared to its corresponding HGS convergence curve. The curve for HGS-DL falls within the range of $[-20, 20]$, indicating improved convergence behavior. Lastly, for the F18 benchmark function, WOA-DL demonstrates convergence within the interval [0.0, 0.1]. This is a notable improvement compared to the convergence reported for WOA, which lies within the range of $[-0.1, 0.3]$. These observations highlight the enhanced convergence behavior achieved by the DL-based hybrid methods for the respective benchmark functions when compared to their base algorithms.

In addition to evaluating the proposed method on classical functions, which were categorized as complex and non-complex, we conducted further assessments on a set of challenging CEC functions. These functions were specifically selected from the CEC01-CEC14 benchmark functions and were designed to incorporate shift (S) and shift rotate (SF) operations. A total of twenty-five (25) CEC functions were included in this evaluation, as outlined in Table 10. These 25 functions were tested against biology-based, evolution-based, and swarm-based metaheuristic algorithms. The inclusion of these challenging functions

Table 13 A comparative analysis of the best solutions obtained for the IEEE CEC 2017 functions for the swarm-based methods, namely AO, FFA, PSO, SSA, HGS, and WOA

ID	AO	AO-DL	FFA	FFA-DL	HGS	HGS-DL	PSO	PSO-DL	SSA	SSA-DL	WOA	WOA-DL
C1	101.8687	100.75	13,865.47	108.1985	110.1544	104.6101	2,462,021	4217.44	100	100	100	100
C2	529.0923	370.7125	4,453.613	2228.617	430.8988	308.066	10,401	29,453.92	200	200	200	200
C3	302.2334	301.7617	6012.8	305.4769	574.2713	305.0815	30,503.17	1285.476	300	300	300	300
C4	400.0004	400.0001	400.7668	400.0046	400	400.003	400.5977	400.575	400	400	400	400
C5	500.0515	500.019	520.0259	500.1283	500.0098	502.317	520	502.777	500	500	500	501.6462
C6	600	600	600.8332	600	600	600	600	600.0031	600	600	600	600
C7	700.0347	700.0016	701.4111	700.1495	700.0224	700.0471	700.4075	700.4134	700	700	700.4064	700.0443
C8	802.0488	800.0001	823.3544	800.0003	800	800.0003	816.8881	800.7085	800	800	802.9849	800
C9	902.3595	900	921.552	900.0001	900.0002	900	924.7373	900.7085	900	900	909.9496	900
C10	1043.061	1000.004	1560.048	1000.053	1000	1000.014	1641.969	1017.182	1000	1000	1000.5	1000
C11	1123.343	1100.011	1636.297	1100.05	1100.003	1100.001	1655.764	1117.182	1100	1100	1435.703	1100
C12	1200.517	1200.067	1200.457	1200.18	1200.65	1200.778	1202.45	1201.28	1200.001	1200	1201.263	1200.407
C13	1300.275	1300.532	1300.175	1300.156	1300.11	1300.512	1300.916	1300.888	1300.13	1300.265	1300.396	1300.516
C14	1400.25	1400.494	1400.256	1400.041	1400.059	1400.35	1400.476	1400.526	1400.5	1400.5	1400.416	1400.497
C15	1500	1500	1500.036	1500	1500	1500	1500.729	1500.13	1500	1500	1500	1500
C16	1600.34	1600.333	1600.404	1600.015	1600.001	1600.348	1600.626	1600.419	1600.411	1600.411	1600.329	1600
C17	1700.263	1700.084	2310.858	1705.003	1700.135	1701.181	12,281.18	1748.958	1700	1700	1700.995	1700.995
C18	1801.476	1800.564	1830.669	1802.078	1800.529	1801.996	1811.123	1803.009	1800.5	1800.5	1804	1801.99
C19	1901	1900.178	1900.3	1900.093	1900.43	1900	1901	1900.284	1900	1900	1901	1900
C20	2013.858	2001.997	3450.43	2011.73	2338.017	2004.168	36,402.52	2131.079	2000.5	2000.5	2233.044	2009.895
C21	2100.09	2100.509	2125.703	2100.829	2100.539	2101.099	2100.361	2101.164	2100.5	2100.5	2102.584	2100.18
C22	2302.988	2302.659	2459.623	2306.944	2303.955	2300.344	2311.269	2385.212	2300.236	2300.236	2300.236	2300.236
C23	2519.977	2400.847	2684.699	2403.653	2401.174	2400.606	2827.114	2461.176	2400	2400	2654.532	2400
C24	2500.49	2501.132	2648.479	2502.226	2501.015	2500.269	2670.476	2553.08	2500	2500	2500.002	2500
C25	2601.807	2602.774	2832.019	2601.707	2602.302	2601.905	2817.598	2693.376	2600	2600	2839.551	2600
Superior	7	16	0	25	11	12	8	17	1	1	3	15
Inferior	16	7	25	0	12	11	17	8	1	1	15	3
Compete	2		0		2		0		23		7	
Overall	16		25		12		17		1		15	

Table 14 Statistical analysis report using Wilcoxon signed-rank test for EOSA, IEOSA, SMA, VCS, IWO, GA, DE, AO, FFA, PSO, SSA, HGS, and WOA

Algorithms	N		Mean rank		Sum of ranks		Z	Significance
	–ve Rank	+ ve Rank	–ve Rank	+ ve Rank	–ve Rank	+ ve Rank		
EOSA-DL—EOSA	11	13	11.64	13.23	128.00	172.00	– .629 ^c	0.530
IEOSA-DL—IEOSA	24	1	13.46	2.00	323.00	2.00	– 4.319 ^b	0.000
IWO-DL—IWO	12	12	11.83	13.17	142.00	158.00	– .229 ^c	0.819
SMA-DL—SMA	11	11	11.91	11.09	131.00	122.00	– .146 ^b	0.884
VCS-DL—VCS	10	15	17.30	10.13	173.00	152.00	– .283 ^b	0.778
GA-DL—GA	21	4	13.86	8.50	291.00	34.00	– 3.458 ^b	0.001
DE-DL—DE	20	5	14.05	8.80	281.00	44.00	– 3.188 ^b	0.001
AO-DL—AO	18	6	13.61	9.17	245.00	55.00	– 2.714 ^b	0.007
FFA-DL—FFA	25	0	13.00	0.00	325.00	0.00	– 4.372 ^b	0.000
HGS-DL—HGS	11	14	16.55	10.21	182.00	143.00	– .525 ^b	0.600
PSO-DL—PSO	19	6	14.26	9.00	271.00	54.00	– 2.919 ^b	0.004
SSA-DL—SSA	1	3	3.00	2.33	3.00	7.00	– .730 ^c	0.465
WOA-DL—WOA	14	4	10.93	4.50	153.00	18.00	– 2.940 ^b	0.003

a. Wilcoxon Signed Ranks Test; **b.** Based on positive ranks; **c.** Based on negative ranks

allowed us to assess the performance and robustness of the proposed method across various algorithm categories.

The evaluation of the 25 CEC functions was conducted on biology-based metaheuristic algorithms, and the results were compared to their corresponding DL-based hybrids, as shown in Table 11. The findings indicate that several DL-based hybrids, including EOSA-DL, IEOSA-DL, SMA-DL, VCS-DL, and IWO-DL, exhibited improvements over their respective base algorithms. While EOSA-DL and IWO-DL demonstrated enhanced performance on 11 and 14 out of the 25 CEC functions, respectively, IEOSA-DL, SMA-DL, and VCS-DL outperformed their base methods on 24, 13, and 9 functions, respectively. The competitive evaluation revealed that SMA-DL and SMA had similar performance on 8 CEC functions.

Examining the overall best biology-based method, the hybrid DL-based approach proved to benefit IEOSA the most. IEOSA-DL showcased improved performance on 24 CEC functions compared to its corresponding classical method. These results highlight the effectiveness of the DL-based hybrid approach in enhancing the performance of biology-based metaheuristic algorithms, particularly about IEOSA, which demonstrated substantial improvements across most of the evaluated CEC 2017 functions. For example, in the case of the C1 function, the IEOSA-DL, IWO-DL, and VCS-DL algorithms demonstrated superior performance compared to an approach utilizing stochastic initialization and a traditional optimizer-related search process. This confirms that the EOSA and SMA methods are stronger without DL support. Additionally, in

C2, EOSA-DL, IEOSA-DL, and SMA-DL showed better performance than their corresponding traditional methods, although IWO and VCS traditional approaches remained stronger than their DL-supported counterparts. We observed that results for C3–C11 indicated improved performance with IEOSA-DL, IWO-DL, and SMA-DL compared to their corresponding traditional approaches, with more comparable performances in other cases. However, the results for the same C3–C11 demonstrated that EOSA and VCS performed better without DL support. These findings suggest that the application of DL support for performance enhancement should not be generalized; instead, it should be considered algorithm-specific.

The evaluation of CEC functions using evolutionary-based algorithms, namely GA and DE, was conducted and compared with their respective DL-supported variants. The results of this comparison are detailed in Table 12. A thorough analysis of the outcomes revealed that DL-supported methods, such as GA-DL and DE-DL, exhibited superior performance compared to their non-DL counterparts, GA and DE, across a significant number of CEC functions. Specifically, GA-DL outperformed GA for 21 functions, while DE-DL outperformed DE for 20 functions. These findings underscore the substantial impact observed when integrating DL-based hybrid methods with evolutionary algorithms.

Table 13 illustrates the assessment of the proposed deep learning hybrid approach when applied to swarm-based algorithms, comparing them with their respective base algorithms. The focus of this comparison is on AO, FFA,

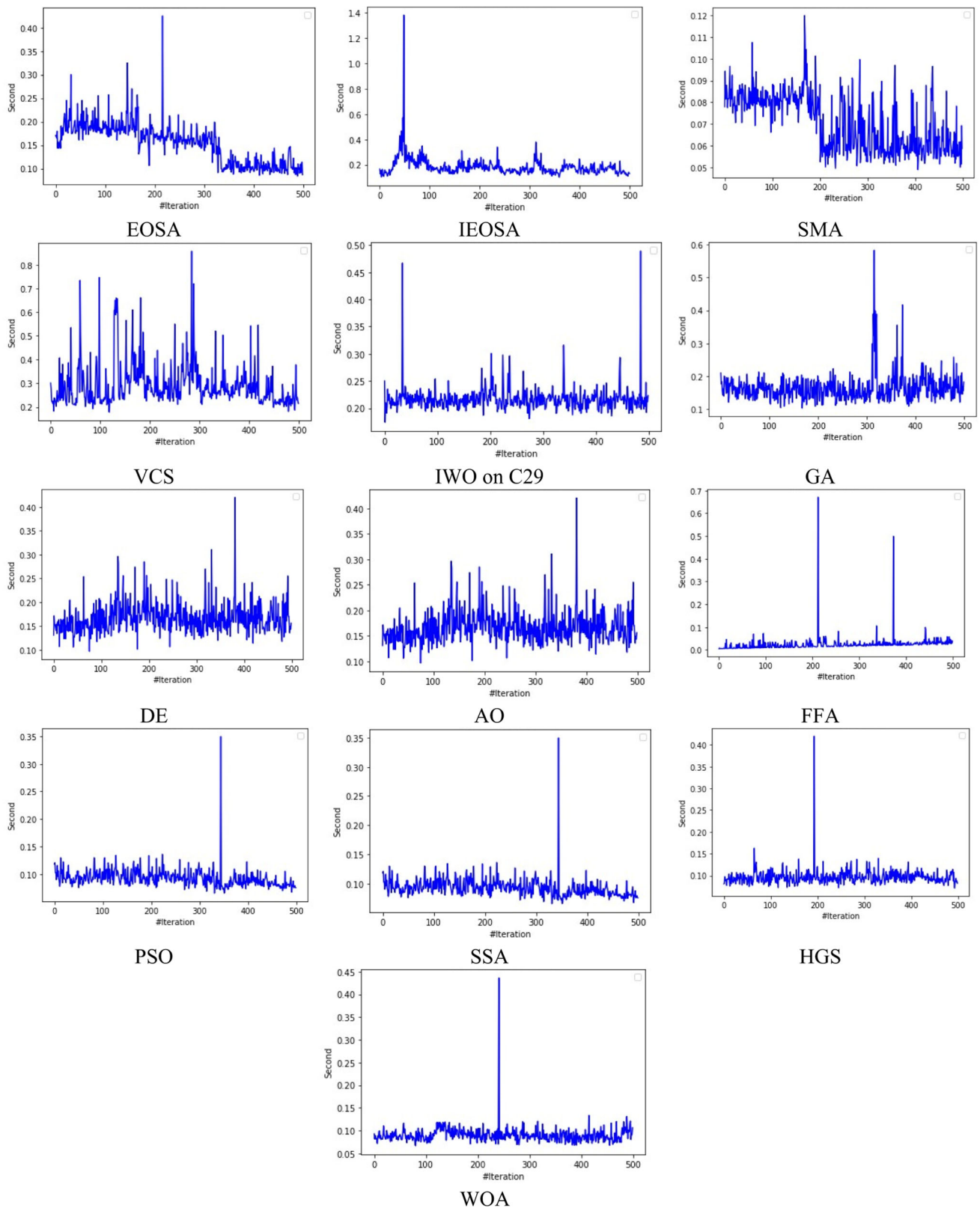


Fig. 12 A comparative analysis of the computational time required for training the EOSA, IEOSA, SMA, VCS, IWO, GA, DE, AO, FFA, PSO, SSA, HGS, and WOA algorithms when investigated using the

CX function. This visual representation offers insights into the relative computational efficiency of each algorithm during the training process

HGS, PSO, SSA, and WOA algorithms, along with their DL-based hybrids, using the CEC functions for evaluation. The results indicate that the DL-based hybrids, including AO-DL, FFA-DL, HGS-DL, PSO-DL, and WOA-DL, consistently outperformed their base algorithms across a significant number of CEC functions. To provide specific instances, AO-DL demonstrated better performance than AO on 16 CEC functions, FFA-DL outperformed FFA on 25 CEC functions, HGS-DL surpassed HGS on 12 CEC functions, PSO-DL improved upon PSO on 17 CEC functions, and WOA-DL exceeded WOA on 15 CEC functions. In contrast, the base algorithms AO, FFA, HGS, PSO, and WOA outperformed their DL-based hybrids in only a few situations. For instance, AO performed better than AO-DL in only 7 CEC functions. Similarly, HGS and PSO were better than their corresponding DL-supported methods in results from only 11 and 8 CEC functions, respectively. Finally, for the WOA methods, the base algorithm outperformed its DL-based hybrid in just 3 CEC functions (C5, C13, and C14). These findings underscore the enhanced optimization performance achieved by hybrid methods. The implication is that the application of the DL-supported approach to the optimization process is more suitable for swarm-based algorithms, similar to what has been observed for evolutionary-based methods. This suggests that swarm and evolutionary methods might share certain characteristics that derive benefit from DL-based solutions.

While SSA-DL does not exhibit improvement over the hybrid method, it demonstrates equivalent performance in 23 CEC functions when compared to the traditional SSA method. Furthermore, despite the few instances where the base methods outperformed AO-DL, FFA-DL, HGS-DL, PSO-DL, and WOA-DL, these cases are minimal, ranging from 2 to 7 CEC functions out of a total of 25. Overall, when evaluating all the hybrids benefiting from the DL-supported approach, FFA-DL emerges with the highest performance, surpassing its corresponding base algorithm in 25 CEC functions. Following closely are PSO-DL, AO-DL, and WOA-DL, with performance improvements observed in 17, 16, and 15 functions, respectively. These findings signify a substantial enhancement in the performance of DL-supported metaheuristic algorithms for both classical and CEC functions. Notably, while significant methodological advancements in enhancing metaheuristic algorithms have often focused on non-AI-driven approaches, the approach and results discussed in this study represent a state-of-the-art application of deep learning for search space initialization and search processes.

The results of the statistical analysis using the Wilcoxon signed-rank test are presented in Table 14 for all the metaheuristic algorithms. The test reveals statistically significant differences in performance between the DL-based

methods and their corresponding base algorithms. When comparing EOSA-DL with EOSA, a median increase in performance between 11.64 and 13.23 is observed, with a z -score of -0.629 and p value of 0.530 . Similarly, EOSA-DL (13.46) shows a significant difference compared to IEOSA (2.00) with a z -score of -4.319 and p value of 0.000 . For IWO-DL, SMA-DL, and VCS-DL, the statistical results also indicate significant performance changes. IWO-DL achieves 11.83 compared to 13.17 for IWO, with a z -score of -0.229 and p value of 0.819 . SMA-DL achieves 11.91 versus 11.09 for SMA, with a z -score of -0.146 and p value of 0.884 . VCS-DL achieves 17.30 versus 10.13 for VCS, with a z -score of -0.283 and p value of 0.778 .

In the case of evolution-based algorithms, GA-DL and DE-DL show statistical improvements in performance compared to their base methods. GA-DL achieves 13.86 versus 8.50 for GA, with a z -score of -3.458 and p value of 0.001 . DE-DL achieves 14.05 versus 8.80 for DE, with a z -score of -3.188 and p value of 0.001 . The Wilcoxon signed-rank test further indicates that AO-DL, FFA-DL, HGS-DL, PSO-DL, SSA-DL, and WOA-DL outperform their corresponding algorithms AO, FFA, HGS, PSO, SSA, and WOA. The mean ranks for the DL-based methods are 13.61, 13.00, 16.55, 14.26, 3.00, and 10.93, respectively, while the mean ranks for the base algorithms are 9.17, 0.00, 10.21, 9.00, 2.33, and 4.50, respectively. The corresponding z -scores and p -values are as follows: AO-DL ($z = -2.714$, $p = 0.007$), FFA-DL ($z = -4.372$, $p = 0.000$), HGS-DL ($z = -0.525$, $p = 0.600$), PSO-DL ($z = -2.919$, $p = 0.004$), SSA-DL ($z = -0.730$, $p = 0.465$), and WOA-DL ($z = -2.940$, $p = 0.003$). These statistical results provide evidence supporting the superior performance of the DL-based metaheuristic algorithms compared to their base counterparts.

In this study, we conducted an investigation into the computational resource requirements and performance of the algorithms considered. The results are presented graphically in Fig. 12. The computational time for EOSA, IEOSA, SMA, VCS, IWO, GA, DE, AO, FFA, PSO, SSA, HGS, and WOA was measured using a complex CX function denoted as $S([C20, C21, C22])$, where S represents a shift function applied to a combination of the C20, C21, and C22 functions.

The runtime of biology-based algorithms, namely EOSA, IEOSA, SMA, VCS, and IWO, was compared on the C30 functions, except for IWO, which was evaluated on the C29 function (which involves a shift on C17, C18, C19 functions). Among these algorithms, EOSA and IWO exhibited significantly higher computational times compared to IEOSA, SMA, and VCS. Interestingly, while IEOSA experienced a peak in computational time around the 50th iteration, it remained consistently low for the

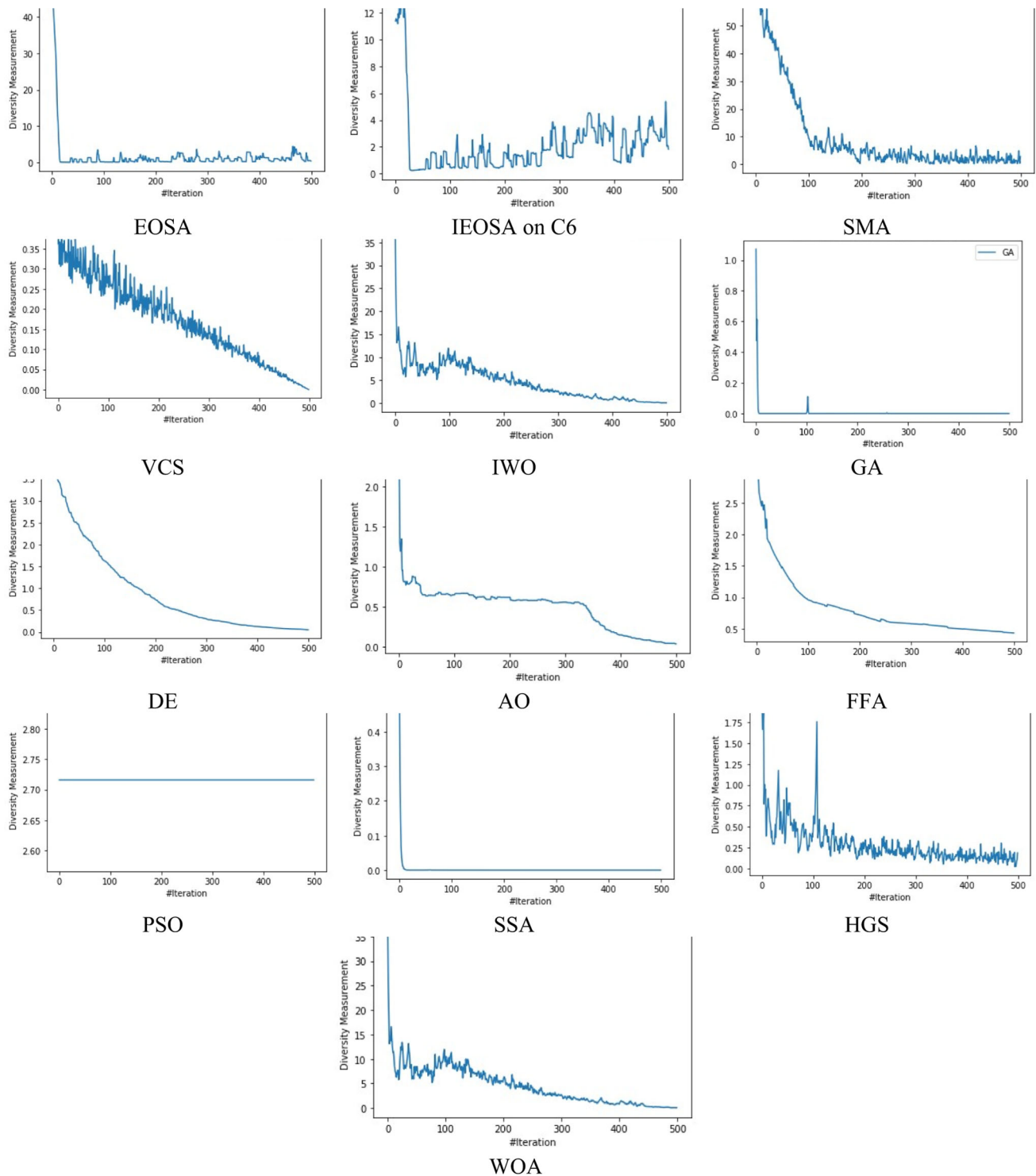


Fig. 13 Measurement of population diversity observed for the EOsa, IEOSA, SMA, VCS, IWO, GA, DE, AO, FFA, PSO, SSA, HGS, and WOA algorithms over 500 iterations. This measurement is demonstrated using the C1 function. The graph provides insights into how

each algorithm maintains population diversity throughout the iterations, which is crucial for assessing their performance and suitability for real-life applications

remaining 450 iterations during training. This remarkable performance of IEOSA, in terms of computational time, in comparison with other biology-based methods indicates

that the latter require substantial computational resources throughout the entire training process. Additionally, a similar low-spike pattern was observed for the EOsa

algorithm from the 200th to the 500th iteration, although its highest computational time was approximately 45 s.

We further compared the impressive performance of the IEOSA algorithm and its variant, the EOSA, with evolution-based and swarm-based methods. Among the evolution-based methods, the GA exhibited a remarkably short computational time, with the highest spike reaching around 0.6 s, while DE required approximately 0.40 s. In contrast, the computational times observed for the swarm-based methods, including AO, FFA, PSO, SSA, HGS, and WOA, showed that AO, FFA, WOA, and HGS consumed significantly more runtime, peaking over 0.40 s. The computational times for PSO and SSA were slightly lower, dropping by 0.05 s to approximately 0.35 s as their highest runtime. This indicates that PSO and SSA are the least computationally expensive algorithms among the swarm-based methods. However, when comparing these two algorithms with the evolution-based GA and biology-based IEOSA, which have the lowest runtimes in their respective categories, it is evident that all the swarm-based algorithms are computationally expensive. Consequently, the biology-based methods exhibit lower computational times compared to all categories of metaheuristic algorithms, with IEOSA emerging as the best-performing algorithm among all those compared.

Additionally, we investigated the algorithms used in this study regarding the quality of their population diversity, which is another important metric for assessing their suitability for real-life problems. Diversity measurements were conducted for all the algorithms, and the results collected over 500 iterations were graphically depicted in Fig. 13. This analysis helps us understand how each algorithm is capable of forming a stable and diverse population.

In the category of biology-based algorithms, namely EOSA, IEOSA, SMA, VCS, and IWO, an investigation was conducted to assess the diversity of their populations. The results revealed that the population of IEOSA quickly converged, leading to lower diversity measurements, while SMA, VCS, IWO, and EOSA exhibited higher diversity measurements. Among the evolution-based algorithms, the GA demonstrated better population diversity compared to DE. In the swarm-based methods, AO, FFA, PSO, SSA, HGS, and WOA, SSA and FFA showed a quicker initial population fit and maintained stability throughout the 500 iterations. Therefore, in terms of diversity measurement, IEOSA, GA, and SSA demonstrated a more diverse population among the algorithms investigated. This indicates that these algorithms were able to avoid premature convergence and maintain a diverse exploration of the solution space.

This section focuses on evaluating the application of DL methods for designing metaheuristic algorithms to enhance their performance on optimization problems. The

performance of LSTM models trained and validated for biology-based, evolution-based, and swarm-based algorithms is discussed. The results demonstrate promising learning curves for all three methods, with the evolution-based algorithms achieving the highest classification accuracy above 0.90, followed by the biology-based algorithms with an accuracy of 0.72. These trained models are then utilized based on their learned patterns to support the design and optimization processes of the investigated metaheuristic algorithms.

A comprehensive comparative analysis is conducted on complex and non-complex benchmark functions for biology-based, evolution-based, and swarm-based algorithms. The findings reveal that metaheuristic algorithms generate valuable dynamic data during the optimization process, which are described through the iteration procedure. This study highlights that these dynamic data have not been effectively utilized to enhance the performance of metaheuristic algorithm design in previous research. Unlike prior studies that mainly focused on creating new algorithms or hybridizing existing ones to form variants, this study introduces the use of deep learning models, particularly the LSTM architecture, to learn significant patterns from the dynamic data generated by optimization algorithms.

The training of LSTM models on this data proves to be effective in supporting the guided method for solution space initialization and search process, leading to improved metaheuristic algorithms. Results obtained from the investigated biology-based, evolution-based, and swarm-based algorithms demonstrate that almost all methods hybridized with DL algorithms exhibit enhanced performance in terms of best solutions and convergence curves. A comparison is made between the best solutions obtained by the original algorithms (e.g., EOSA, IEOSA, SMA, VCS, etc.) and their corresponding DL-based hybrids (e.g., EOSA-DL, IEOSA-DL, SMA-DL, VCS-DL, etc.). The DL-based hybrids consistently outperform the classical optimization approaches, establishing their superiority and overall better performance.

Furthermore, the convergence trajectories of selected individuals in the solution space across 500 iterations for all metaheuristic algorithms are plotted and compared. The comparative analysis of these graphs further confirms the benefits of applying the deep learning-based method to improve the optimization process. The study successfully achieves its objective and advances research in the domain by demonstrating the effectiveness of DL-based optimization algorithms.

6 Conclusion and future work

This paper introduces a novel method aimed at enhancing the design and optimization process of metaheuristic algorithms. The proposed approach utilizes deep learning algorithms to improve solution space initialization and optimization by means of a guided search using deep learning models. The performance of this method is compared with existing biology-based, evolution-based, and swarm-based metaheuristic algorithms, including EOSA, IEOSA, SMA, VCS, IWO, GA, DE, AO, FFA, PSO, SSA, HGS, and WOA. Additionally, an LSTM architecture is developed and trained using dynamically collected datasets obtained during the execution of the aforementioned optimization algorithms. These trained models are then employed to enhance the design and optimization processes of all 13 algorithms considered in the study. The collected results are evaluated, and the comparative analysis demonstrates that the deep learning-based metaheuristic algorithms outperform classical optimization approaches. This highlights the significance of incorporating deep learning or deep learning models to achieve enhanced performance in the field, rather than solely focusing on developing new optimization algorithms or hybrid variants.

The findings of this study illustrate the potential of the proposed deep learning-based metaheuristic design approach in solving combinatorial optimization problems. The training of deep learning models heavily relies on training data, enabling them to make accurate predictions or generate outputs when required. In this study, the focus was on training deep learning models for a generative task, specifically to generate an initial solution space for metaheuristic algorithms. The objective was to surpass existing methods and establish the proposed approach as a leading advancement in the field. Encouragingly, the experimental results validated this hypothesis, demonstrating the effectiveness of the guided search approach. The proposed method consistently outperformed alternative techniques, showcasing its ability to swiftly identify superior solutions.

While this study specifically focused on biology-based, evolution-based, and swarm-based algorithms, future research could explore the potential benefits of the proposed method for other metaheuristic algorithms. Furthermore, instead of categorization-based training of the LSTM model, it would be worthwhile to investigate algorithm-specific neural network training to optimize the results of each individual metaheuristic algorithm, as the learning process would be tailored to the optimization patterns of the selected algorithm. Expanding the dynamic data collected for training the deep learning model to include a wider range of benchmark functions would also contribute

to a more comprehensive dataset, ensuring the generalization of the trained model. Additionally, increasing the number of training epochs beyond the 50 epochs used in this study should be considered to further improve the model's generalization. For future studies, employing a word-level input for the deep learning model, as opposed to the character-level input used in this study, may yield additional improvements. Moreover, it would be both intriguing and essential to broaden the scope of the current study by integrating sets of real-time data.

Author contributions ONO and AEE wrote the main manuscript text, ONO prepared Figs. 1–13, while AEE supervised and edited the initial draft of the main manuscript. Subsequently, all authors collectively reviewed and made revisions to the manuscript.

Funding Open access funding provided by North-West University.

Data availability statements All data generated or analyzed during this study are included in this article.

Declarations

Conflict of interest The authors declare that there is no conflict of interest with regard to the publication of this paper.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Raziani S, Azimbagirad M (2022) Deep CNN hyperparameter optimization algorithmss for sensor-based human activity recognition. *Neurosci Inform* 2(3):100078
2. Oyelade O, Ezugwu AE-S (2021) Characterization of abnormalities in breast cancer images using nature-inspired metaheuristic optimized convolutional neural networks model. *Concurr Comput Pract Exp* 34(4):1–32
3. Oyelade ON, Ezugwu AE (2021) A bioinspired neural architecture search based convolutional neural network for breast cancer detection using histopathology images. *Sci Rep* 11(1):1–28
4. Cagnina L, Esquivel S, Coello CAC (2008) Solving engineering optimization problems with the simple constrained particle swarm optimizer. *Informatica* 32(3):319–326
5. Tang J, Liu G, Pan Q (2021) A review on representative swarm intelligence algorithms for solving optimization problems: applications and trends. *IEEE/CAA J Autom Sin* 8(10):1627–1643

6. Olaide Oyelade N, Absalom Ezugwu E (2020) A deep learning model using data augmentation for detection of architectural distortion in whole and patches of images. *Biomed Signal Process Control* 65(2021):1–17
7. Oyelade O, Ezugwu AE-S (2022) A comparative performance study of random-grid model for hyperparameters selection in detection of abnormalities in digital breast images. *Concurr Comput Pract Exp* 34(13):1–32
8. Zhu F, Ye F, Fu Y, Liu Q, Shen B (2019) Electrocardiogram generation with a bidirectional LSTM-CNN generative adversarial network. *Sci Rep* 9(6734):1–11
9. Dhali I, Vashisth S, Saraswat S (2020) Text generation using long short-term memory networks. *Micro-Electron Telecommun Eng Lect Notes Netw Syst* 106(106):649–657
10. Olaide O, Ezugwu AE-S (2022) A novel wavelet decomposition and transformation convolutional neural network with data augmentation for breast cancer detection using digital mammogram. *Sci Rep* 12(1):1–30
11. Oyelade ON, Ezugwu AE (2021) ArchGAN: a generative adversarial network for architectural distortion abnormalities in digital mammograms. In: 2021 International conference on electrical, computer and energy technologies (ICECET)
12. Rehmer A, Kroll A (2020) On the vanishing and exploding gradient problem in gated recurrent units. *IFAC-papersOnline* 51(2):1243–1248
13. Holland JH (1992) Genetic algorithms. *Sci Am* 267(1):66–73
14. Karaboga D (2005) An idea based on honey bee swarm for numerical optimization, vol 200. Technical report-tr06, Erciyes University, Engineering Faculty, Computer Engineering Department, pp 1–10
15. Eberhart R, Kennedy J (1995) A new optimizer using particle swarm theory. In: MHS'95. Proceedings of the sixth international symposium on micro machine and human science. IEEE, pp 39–43
16. Yang X, Deb S (2009) Cuckoo search via Lévy flights, “ In: 2009 World congress on nature and biologically inspired computing (NaBIC)
17. Abualigah L, Diabat A, Mirjalili S, Abd Elaziz M, Gandomi AH (2021) The arithmetic optimization algorithm. *Comput Methods Appl Mech Eng* 376:113609
18. Yang Y, Chen H, Heidari AA, Gandomi AH (2021) Hunger games search: visions, conception, implementation, deep analysis, perspectives, and towards performance shifts. *Expert Syst Appl* 177:114864
19. Nguyen T, Nguyen G, Nguyen BM (2020) EO-CNN: an enhanced CNN model trained by equilibrium optimization for traffic transportation prediction. *Procedia Comput Sci* 176:800–809
20. Oyelade ON, Ezugwu AE (2022) Ebola optimization search algorithm (EOSA): a new metaheuristic algorithm based on the propagation model of Ebola virus disease. *IEEE Access* 10:1–38
21. Oyelade ON, Ezugwu AE (2021) Ebola optimization search algorithm (EOSA): a new metaheuristic algorithm based on the propagation model of Ebola virus disease. In: International conference on electrical, computer and energy technologies—ICECET, Cape Town. IEEE
22. Oyelade ON, Ezugwu AE-s (2022) Immunity-based ebola optimization search algorithm (IEOSA) for minimization of feature extraction with reduction in digital mammography using CNN models. *Sci Rep* 13(1)
23. Talbi E-G (2021) Machine learning into metaheuristics: a survey and taxonomy of data-driven metaheuristics. *ACM Comput Surv* 54(6):1–32
24. Zennaki M, Cherif AE (2010) A new machine learning based approach for tuning metaheuristics for the solution of hard combinatorial optimization problems. *J Appl Sci* 10:1–10
25. Lessmann S, Caserta M, Arango IM (2011) Tuning metaheuristics: a data mining based approach for particle swarm optimization. *Expert Syst Appl* 38(2011):12826–12838
26. Huang C, Li Y, Yao X (2020) A survey of automatic parameter tuning methods for metaheuristics. *IEEE Trans Evol Comput* 24(2):201–217
27. Souza MD (2021) Automatic design of heuristic algorithms for binary optimization problems. In: Proceedings of the thirtieth international joint conference on artificial intelligence (IJCAI-21)
28. Pessoa LFD, Hellingrath B, Neto FBDL (2019) Automatic generation of optimization algorithms for production lot-sizing problems. In: 2019 IEEE congress on evolutionary computation (CEC)
29. Fan C, Hou B, Zheng J, Xiao L, Yi L (2020) A surrogate-assisted particle swarm optimization using ensemble learning for expensive problems with small sample datasets. *Appl Soft Comput J* 91(2020):1–17
30. Liu N, Pan J-S, Chu CSS-C (2020) An efficient surrogate-assisted quasi-affine transformation evolutionary algorithm for expensive optimization problems. *Knowl Based Syst* 209(2020):1–12
31. Oliveira J, Almeida M, Santos R, de Gusmão R, Brito A (2020) New surrogate approaches applied to meta-heuristic algorithms. In: Artificial intelligence and soft computing. ICAISC 2020. Lecture notes in computer science
32. Tao J, Sun G (2019) Application of deep learning based multi-fidelity surrogate model to robust aerodynamic design optimization. *Aerospace Sci Technol* 92:722–737
33. Jiang M, Huang Z, Qiu L, Huang W, Yen GG (2018) Transfer learning-based dynamic multiobjective optimization algorithms. *IEEE Trans Evol Comput* 22(4):501–514
34. Al-Obeidat F, Belacel N, Spencer B (2019) Combining machine learning and metaheuristics algorithms for classification method PROAFTN. In: Enhanced living environments, LNCS 11369
35. Gutierrez-Rodríguez AE, Conant-Pablos SE, Ortiz-Bayliss JC, Terashima-Marín H (2019) Selecting meta-heuristics for solving vehicle routing problems with time windows via meta-learning. *Expert Syst Appl* 118(2020):470–481
36. Li S, Chen H, Wang M, Heidari AA, Mirjalili S (2020) Slime mould algorithm: a new method for stochastic optimization. *Futur Gener Comput Syst* 111:300–323
37. Li MD, Zhao H, Weng XW, Han T (2016) A novel nature-inspired algorithm for optimization: virus colony search. *Adv Eng Softw* 92:65–88
38. Mehrabian AR, Lucas C (2006) A novel numerical optimization algorithm inspired from weed colonization. *Eco Inform* 1:355–366
39. Storn R, Price K (1997) Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *J Glob Optim* 11(4):341–359
40. Abualigah L, Yousri D, Abd Elaziz M, Ewees AA, Al-qaness MA, Gandomi AH (2021) Aquila optimizer: a novel metaheuristic optimization algorithm. *Comput Ind Eng* 157:107250
41. Łukasik S, Żak S (2009) Firefly algorithm for continuous constrained optimization tasks. In: International conference on computational collective intelligence. Springer, Berlin, Heidelberg
42. Xue J, Shen B (2020) A novel swarm intelligence optimization approach: sparrow search algorithm. *Syst Sci Control Eng* 8(1):22–34
43. Mirjalili S, Lewis A (2016) The whale optimization algorithm. *Adv Eng Softw* 95:51–67
44. Karimi-Mamaghana M, Mohammadi M, Meyer P, Karimi-Mamaghana AM, Talbi E-G (2022) Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: a state-of-the-art. *Eur J Oper Res* 296(2022):393–422

45. Kazimipour B, Li X, Qin AK (2014) A review of population initialization techniques for evolutionary algorithms. In: 2014 IEEE congress on evolutionary computation (CEC) July 6–11, 2014, Beijing, China
46. Ashraf A, Pervaiz S, Bangyal WH, Nisar K, Ibrahim AAA, Rodrigues JJPC, Rawat DB (2021) Studying the impact of initialization for population-based algorithms with low-discrepancy sequences. *Appl Sci* 11:8190
47. Pan W, Li K, Wang M, Wang J, Jiang B (2014) Adaptive randomness: a new population initialization method. *Math Prob Eng* 2014:1–14
48. Keedwell E, Brevilliers M, Idoumghar L, Lepagnot J, Rakhshani H (2018) A novel population initialization method based on support vector machine, Miyazaki, Japan
49. Agushaka JO, Ezugwu AE (2022) Initialisation approaches for population-based metaheuristic algorithms: a comprehensive review. *Appl Sci* 12:896
50. Tharwat A, Schenck W (2021) Population initialization techniques for evolutionary algorithms for single-objective constrained optimization problems: deterministic vs. stochastic techniques. *Swarm Evolut Comput* 67(2021):100952
51. Hughes M, Goerigk M, Dokka T (2021) Automatic generation of algorithms for robust optimisation problems using grammar-guided genetic programming. *Comput Oper Res* 133:2021
52. Nakib A, Hilia M, Heliodore F, Talbi E-G (2017) Design of metaheuristic based on machine learning: a unified approach. In: 2017 IEEE international parallel and distributed processing symposium workshops
53. Dobslaw F (2010) A parameter tuning framework for metaheuristics based on design of experiments and artificial neural networks. In: Proceedings of the international conference on computer mathematics and natural computing
54. Garmendia AI, Ceberio J, Mendiburu A (2022) Neural combinatorial optimization: a new player in the field, pp 1–12. [arXiv:2205.01356v1](https://arxiv.org/abs/2205.01356v1)
55. Rahati A, Rakhshani H (2016) A gene expression programming framework for evolutionary design of metaheuristic algorithms. In: 2016 IEEE congress on evolutionary computation (CEC)
56. Mejía-de-Dios J-A, Mezura-Montes E (2020) A surrogate-assisted metaheuristic for bilevel optimization. In: GECCO'20, Cancún, Mexico
57. Chen M, Chen Y, Du Y, Wei L, Chen Y (2020) Heuristic algorithms based on deep reinforcement learning for quadratic unconstrained binary optimization. *Knowl Based Syst* 204:106366
58. Sun J, Zhang H, Zhou A, Zhang Q, Zhang K (2019) A new learning-based adaptive multi-objective evolutionary algorithm. *Swarm Evol Comput* 44(2019):304–319
59. Wauters T, Verbeeck K, Causmaecker PD, Berghe GV (2013) Boosting metaheuristic search using reinforcement learning. In: Hybrid metaheuristic: studies in computational intelligence, pp 433–452
60. Calvet L, Armas JD, Masip D, Juan AA (2017) Learnheuristics: hybridizing metaheuristics with machine learning for optimization with dynamic inputs. *Open Math* 15:261–280
61. Vij P, Nikam S, Bhatia A (2020) Detection of algorithmically generated domain names using LSTM. In: 2020 12th international conference on communication systems and networks (COMSNETS)
62. Wen T-H, Gasic M, Mrksic N, Su P-H, Vandyke D, Young S (2015) Semantically conditioned LSTM-based natural language generation for spoken dialogue systems. In: Proceedings of the 2015 conference on empirical methods in natural language processing
63. Santhanam S (2020) Context based text-generation using LSTM networks, pp. 1–11. [arXiv:2005.00048v1](https://arxiv.org/abs/2005.00048v1)
64. Islama S, Mousumia SSS, Abujarb S, Hossain SA (2019) Sequence-to-sequence bangla sentence generation with LSTM recurrent neural networks. *Procedia Comput Sci* 152:51–58
65. Cai X, Songa B, Fang Z (2019) Exemplar based regular texture synthesis using LSTM. *Pattern Recogn Lett* 128(2019):226–230
66. Rauf HT, Gao J, Almadhor A, Arif M, Nafis T (2021) Enhanced bat algorithm for COVID-19 short-term forecasting using optimized LSTM. *Soft Comput* 25:12989–12999
67. Bouktif S, Fiaz A, Ouni A, Serhani MA (2020) Multi-sequence LSTM-RNN Deep learning and metaheuristics for electric load forecasting. *Energies*, MDPI 13(391):1–21
68. Alkabbani H, Ahmadian A, Zhu Q, Elkamel A (2021) Machine learning and metaheuristic methods for renewable power forecasting: a recent review. *Front Chem Eng* 3(665415):1–21
69. Kumar K, Haider TU (2020) Enhanced prediction of intra-day stock market using metaheuristic optimization on RNN–LSTM Network. *N Gener Comput* 39(10):1–42
70. Kumar P, Batra S (2018) Meta-heuristic based optimized deep neural network for streaming data prediction. In: International conference on advances in computing, communication control and networking (ICACCCN2018)
71. Conner M, Gral L, Adams K, Hunger D, Strelow R, Neuwirth A (2022) Music generation using an LSTM. [arXiv:2203.12105](https://arxiv.org/abs/2203.12105)
72. Qiao Y-L, Lai Y-K, Fu H, Gao L (2022) Synthesizing mesh deformation sequences with bidirectional LSTM. *IEEE Trans Visualiz Comput Graph* 28(4):1906–1916
73. Oyelade O, Ezugwu AE-S, Alumatri M, Saha AK, Abualigah L, Chiroma H (2022) A generative adversarial network for synthesis of regions of interest based on digital mammograms. *Sci Rep* 12(1):1–32

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.